Chip Multiprocessors and Vector Computers

Agenda

- Introduction
- Simultaneous multithreading
- Chip multiprocessors
  - Overview
  - Memory hierarchy
  - Thread scheduling
- Vector computers
  - Vector processing in depth
  - SIMD engines in modern processors

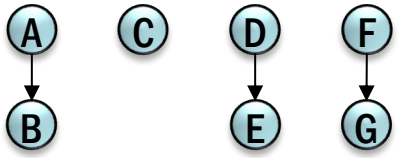
Introduction

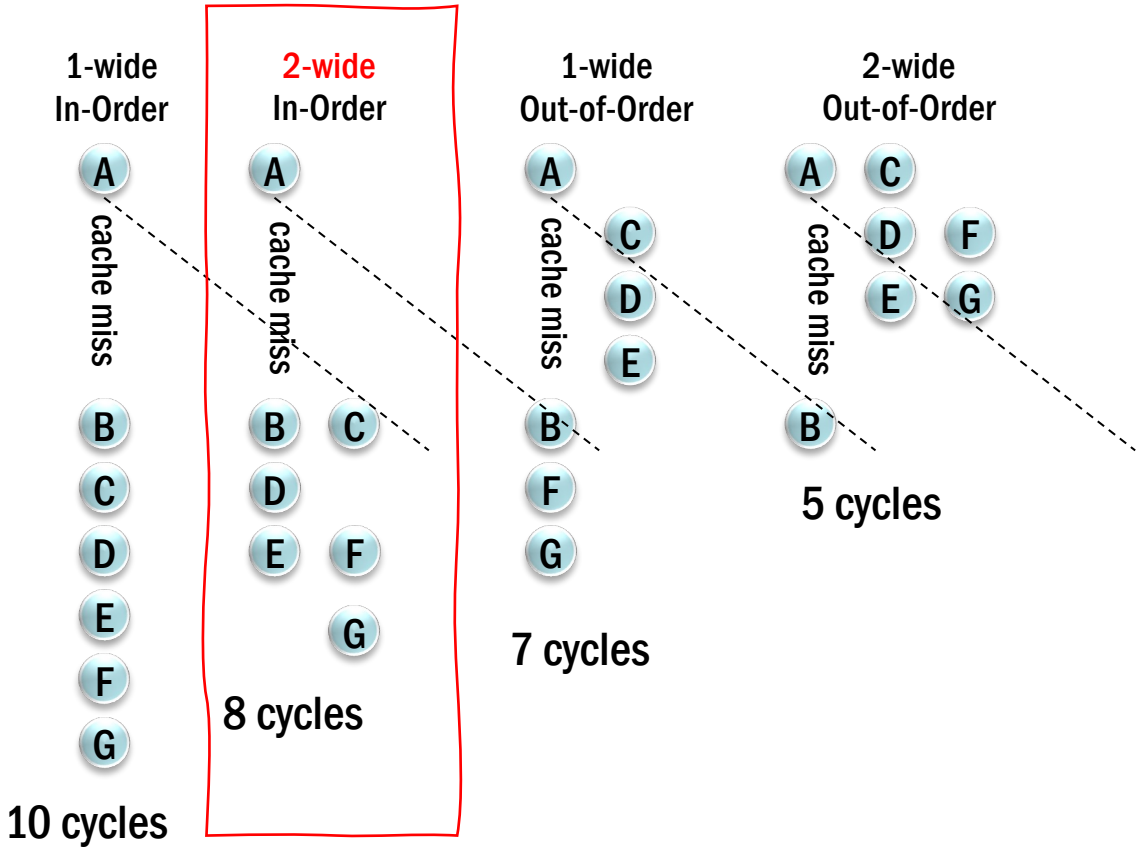
- Growing transistor budgets
  - Modern processors integrate billions of transistors
- Diminishing returns from deeper pipelines
  - Pipeline overhead and branch mis-prediction penalties limit further gains
- Higher frequencies and larger out-of-order scheduling structures increase power consumption and design complexity
- Conventional dynamic-issue superscalar approach
  - ILP limited in single thread of control
  - Large issue window → more complexity, less returns
- Optimal usage of silicon space needed → Exploit DLP and TLP
  - Simultaneous multithreading (SMT)
  - Chip multiprocessor (CMP)

Recap: Superscalar and Out-of-Order Pipelines

- Superscalar: executing multiple instructions in parallel
  - E.g., 2-wide, 4-wide issues
  - Multi-issue front-end, multiple functional units (ALUs, FPUs, LD/ST units)

A: R1 = Load 16[R2]
B: R3 = R1 + R4
C: R6 = Load 8[R9]
D: R5 = R2 – 4
E: R7 = Load 20[R5]
F: R4 = R4 – 1
G: BEQ R4, #0

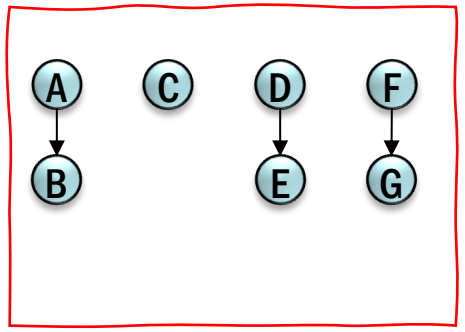


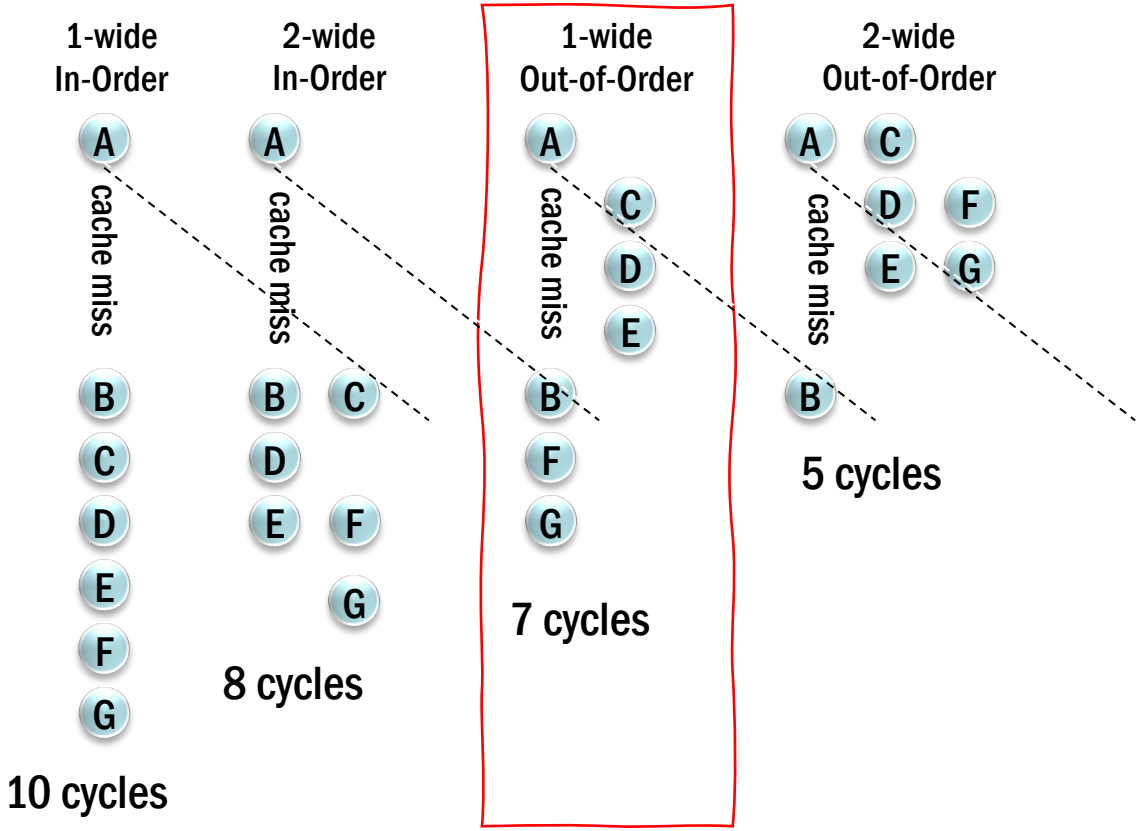


Recap: Superscalar and Out-of-Order Pipelines

- Out-of-order execution: dynamically determines which instructions to execute
  - Execute instructions as soon as register and memory dependencies are known
  - Latency tolerance: allow independent instructions to execute and complete in the presence of a long latency operation
  - Execute out-of-order, retire (commit) instructions in order

A: R1 = Load 16[R2]
B: R3 = R1 + R4
C: R6 = Load 8[R9]
D: R5 = R2 – 4
E: R7 = Load 20[R5]
F: R4 = R4 – 1
G: BEQ R4, #0

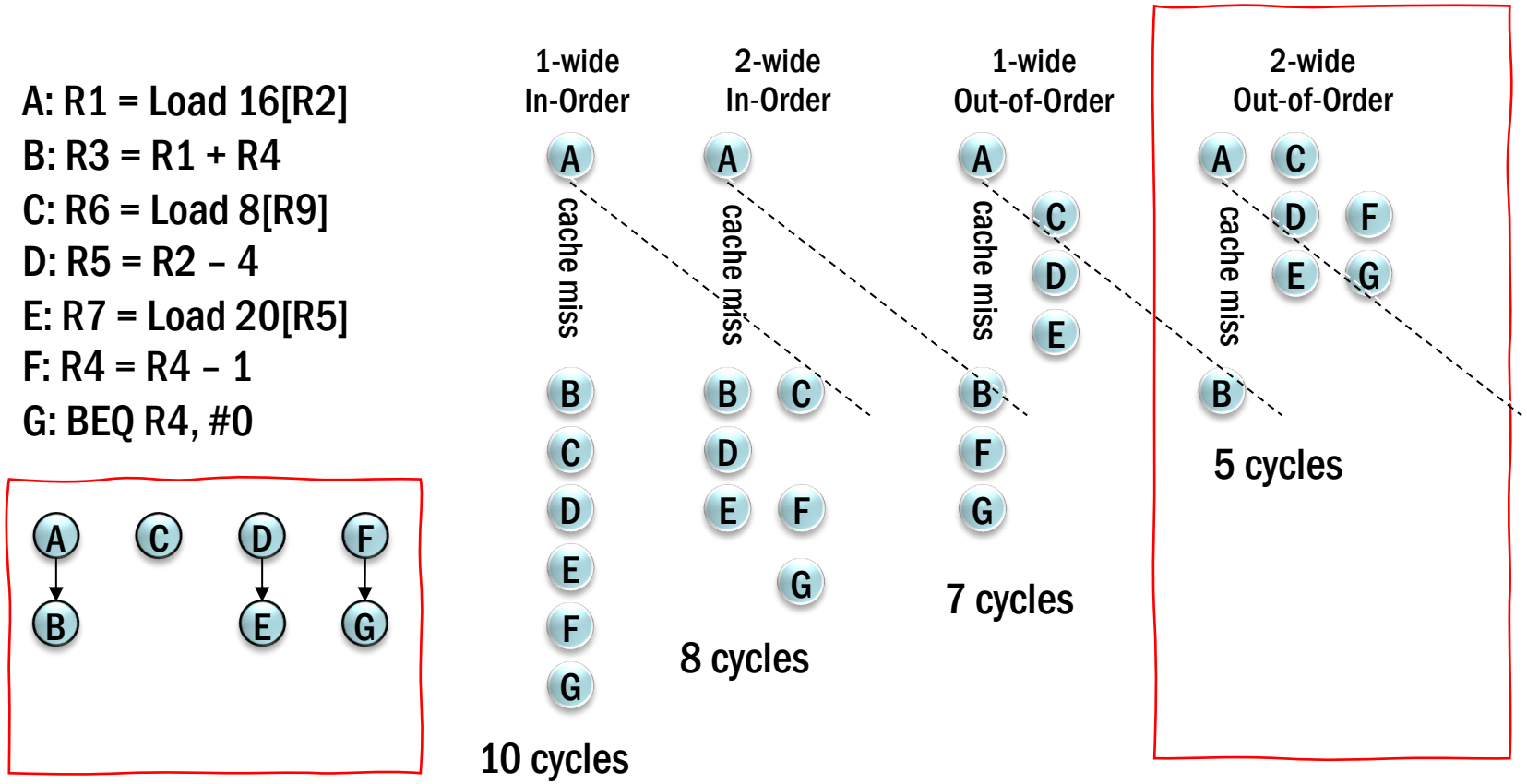




Recap: Superscalar and Out-of-Order Pipelines

◎ Superscalar and out-of-order execution: dynamically determines which instructions to execute (multiple per cycle)

- Dynamic instruction scheduling + instruction-level parallelism

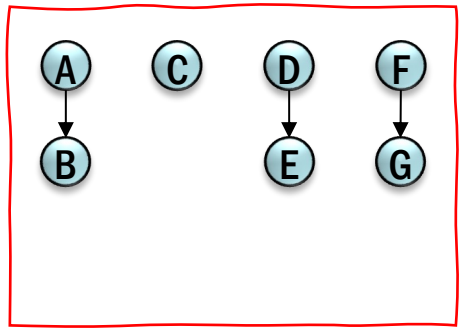


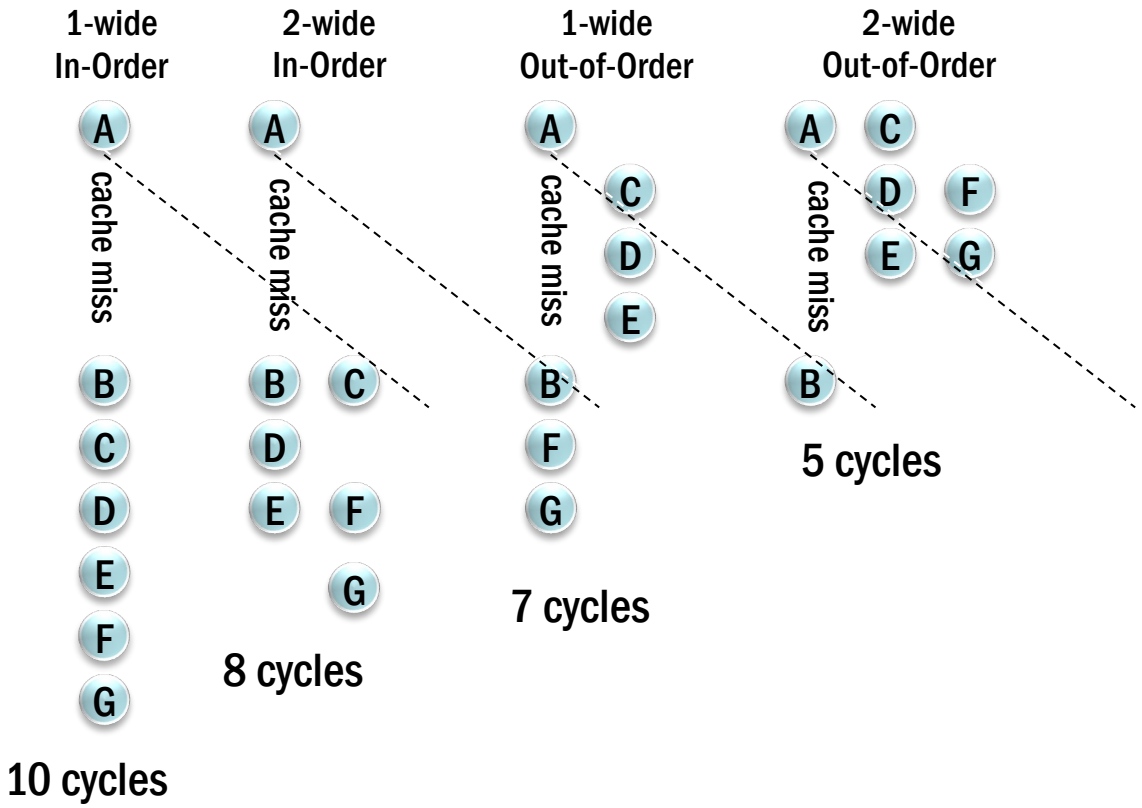
Recap: Superscalar and Out-of-Order Pipelines

◎ Speculative Execution: executes instructions before it is certain they are required and safe to execute them

- E.g., branch prediction, memory disambiguation

A: R1 = Load 16[R2]
B: R3 = R1 + R4
C: R6 = Load 8[R9]
D: R5 = R2 – 4
E: R7 = Load 20[R5]
F: R4 = R4 – 1
G: BEQ R4, #0
H: #0 R8 = R5 + R6





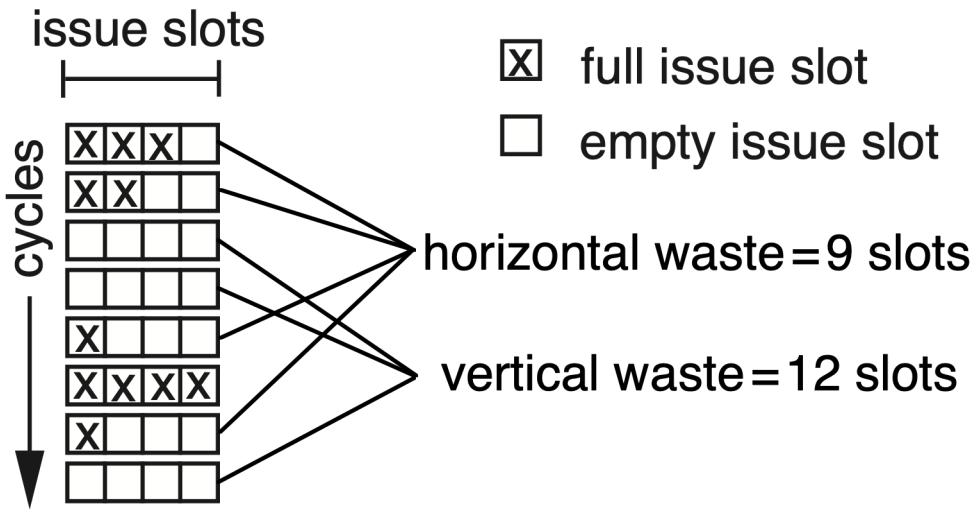
Hardware Multithreading

◎ Find a way to “hide” true data dependency stalls, cache miss stalls, and branch stalls by finding instructions (from other process threads) that are independent of those stalling instructions

◎ Multithreading increases the utilization of resources on a chip by allowing multiple processes (threads) to share the functional units of a single processor

- Processor must duplicate the state hardware for each thread – a separate register file, PC, instruction buffer, and store buffer for each thread
- The caches, buffers can be shared (although the miss rates may increase if they are not sized accordingly)
- The memory can be shared through virtual memory mechanisms
- Hardware must support efficient thread context switching

Superscalar and Fine-grained Multithreading



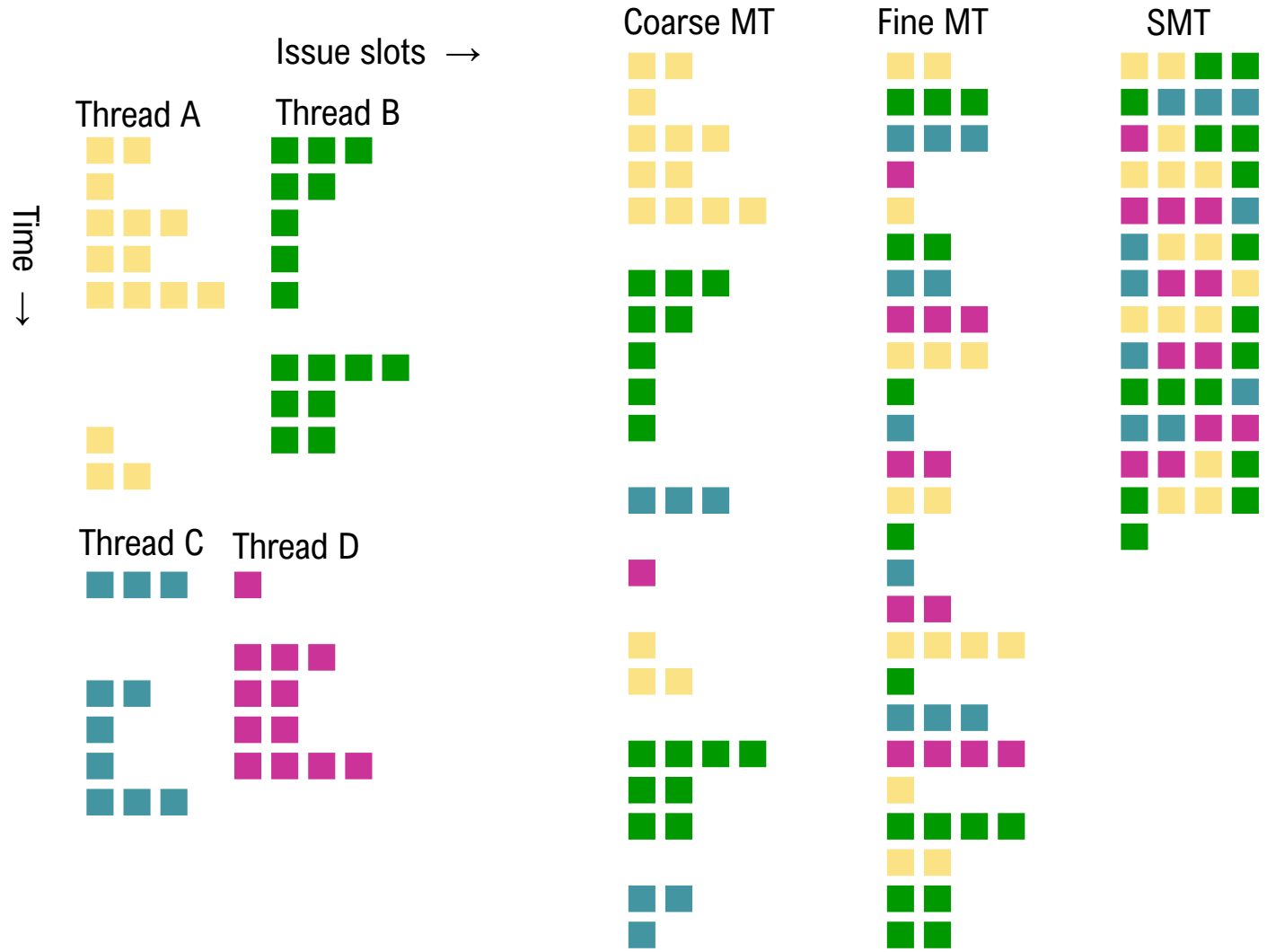
- Horizontal Waste: Cannot issue as many instructions as issue slots
- Vertical Waste: Processor can't issue any instructions in a cycle
- Superscalar: Both horizontal and vertical waste
- Fine-grain multithreading: Can eliminate some of the vertical waste

Simultaneous Multithreading

- Objectives
  - Speedup one application (parallel)
  - Speedup a mix of serial applications (throughput)
- Key idea: Attack both horizontal and vertical waste
  - Multiple threads share functional units and issue slots in the same cycle
- Advantages
  - Can utilize all the resources (less waste)
  - Can run single applications faster
- Disadvantages
  - More complicated design: FU, issue slots are shared
  - Wire delays kill: slower frequency

Dean M. Tullsen, Susan J. Eggers, and Henry M. Levy. Simultaneous multithreading: maximizing on-chip parallelism. In ISCA '95

Example: Threading on a 4-way SS Processor



CMP Motivation

- Limits of wide superscalar cores
  - Modern IPC gains can require lower clock rates, higher power, and substantially greater design and verification effort

- **CMP approach: several moderately wide cores**
  - Exploit TLP across cores
  - Retain practical ILP within each core

- Technology push
  - Benefits of wide issue are limited
  - Decentralized microarchitecture: easier to build several simple fast processors than one complex processor

- Application pull
  - Applications expose parallelism at different granularities
  - Available ILP varies widely
    - < 10 instructions/cycle (INT applications)
    - > 40 instructions/cycle (loops in FP applications)

Microarchitectures Compared

- Superscalar: 6-way superscalar (@500 MHz)
- CMP: Four 2-way superscalars (@500 MHz)

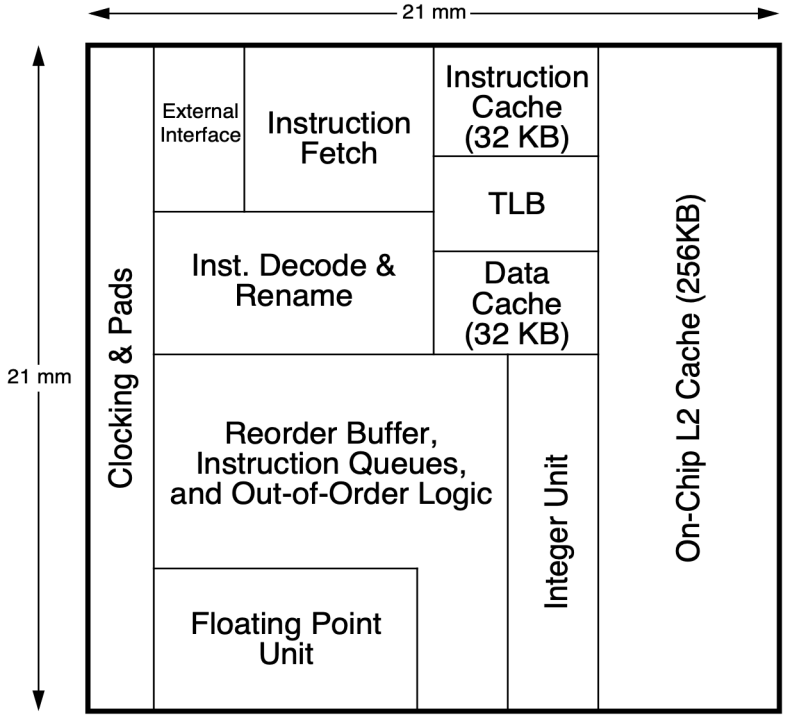


Figure 2. Floorplan for the six-issue dynamic superscalar microprocessor.

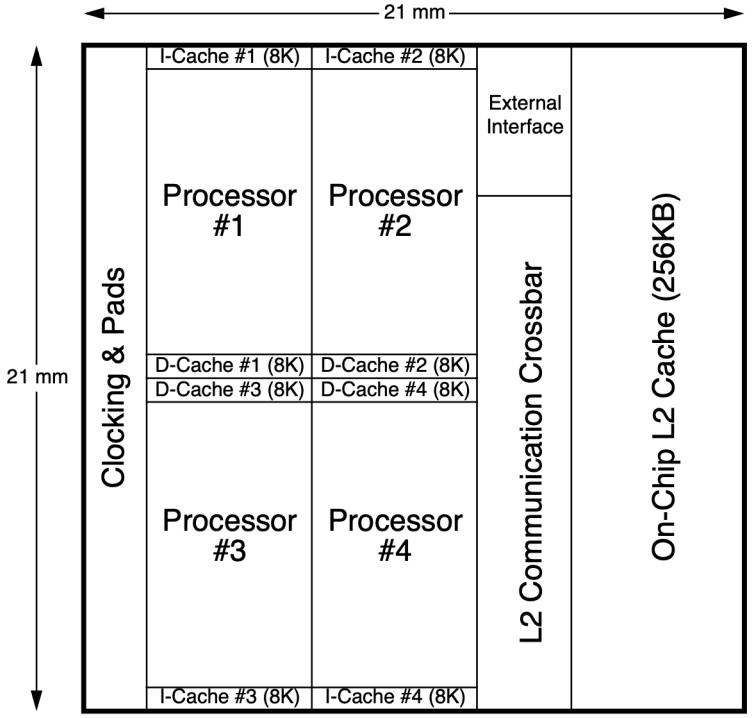
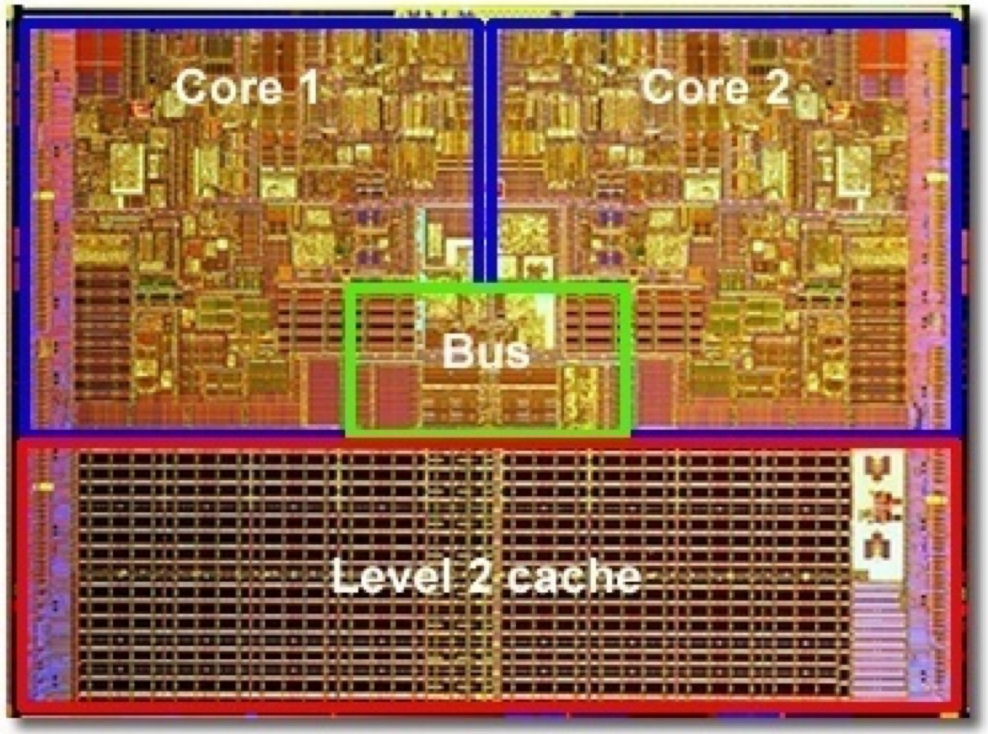


Figure 3. Floorplan for the four-way single-chip multiprocessor.

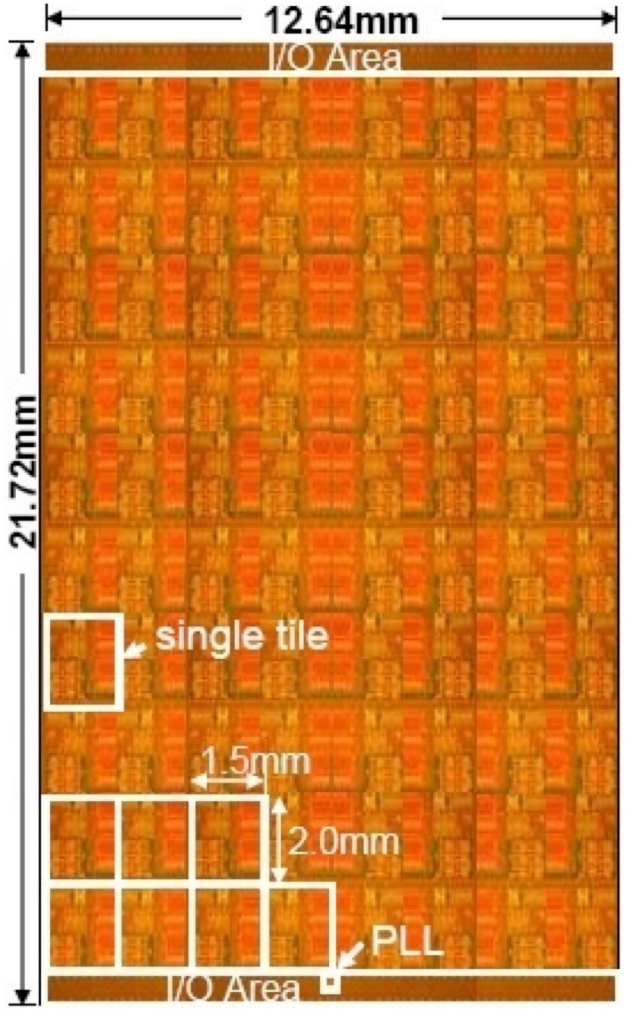
Early CMP's

- Intel Core Duo (2006)
  - 2 cores
    - 12-stage pipeline
    - 2-way SMT (HT)
    - Up to 2.33 GHz
    - P6 (Pentium M) microarchitecture
  - 2MB shared L2 cache
  - 151M transistors in 65nm technology
  - Power consumption between 9W and 30W



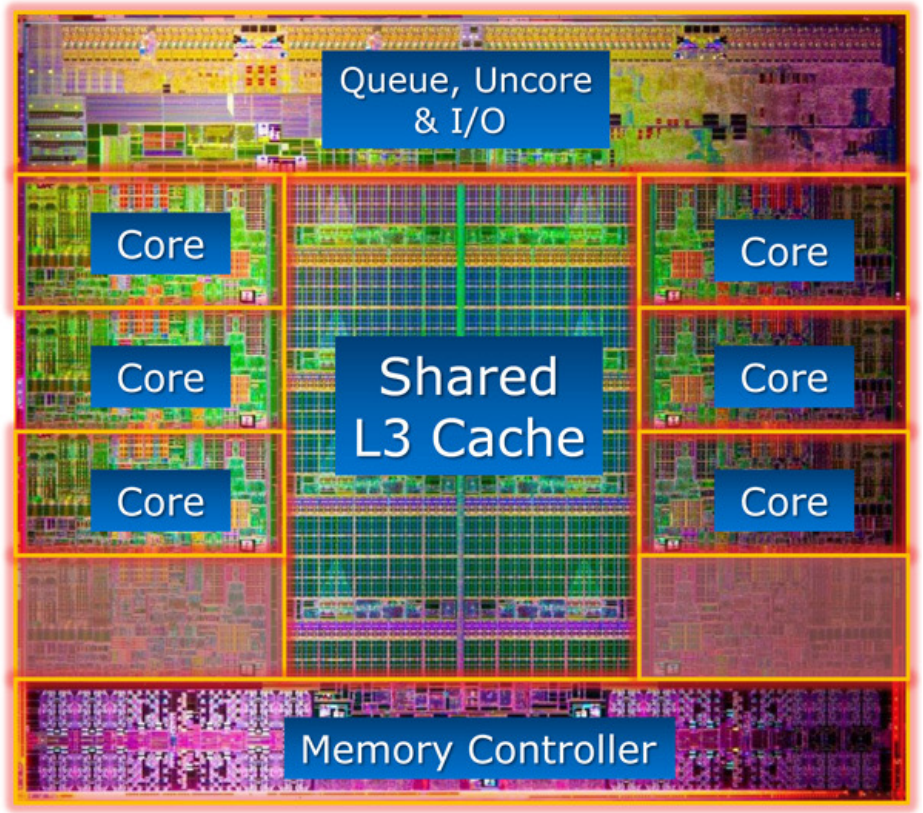
Different Design for CMP

- Many-core architectures
- Intel Polaris (2007)
  - 80 cores
    - Single issue, statically scheduled
    - 3.2 GHz (up to 5 GHz)
  - No shared L2 or L3 cache
  - “Tiled” approach: Core + cache + router
  - Scalable, packet-switched interconnect (8x10 mesh)
  - Power consumption around 62W
- Intel SCC (Single-Chip Cloud Computer) (2010)
  - 48 cores
  - No hardware cache coherence (message-passing mesh network)



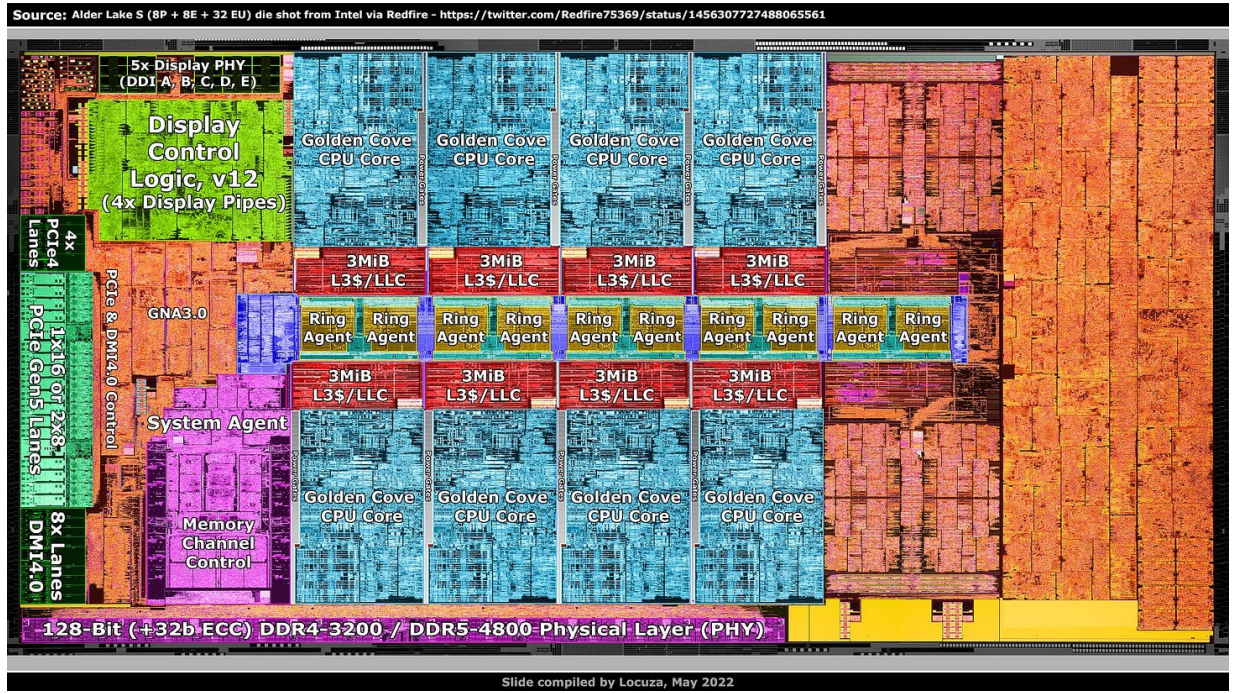
More Examples

- Intel Core i7 (2012)
  - 2,4,6,8 cores
    - 2-way SMT (HT)
    - Up to 3.5 GHz
    - Sandybridge microarchitecture
  - Up to 20MB shared L3
  - Up to 2B transistors in 22nm technology
  - Power consumption between 45W and 150W



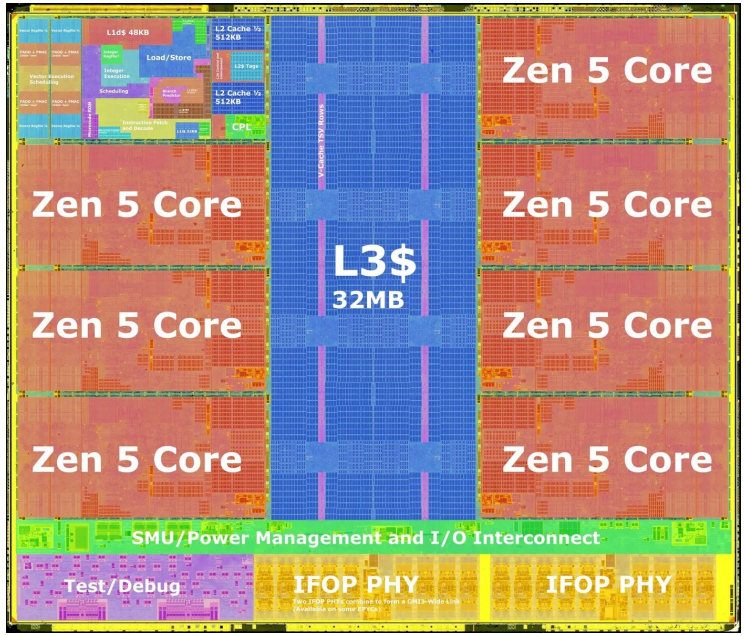
More Examples

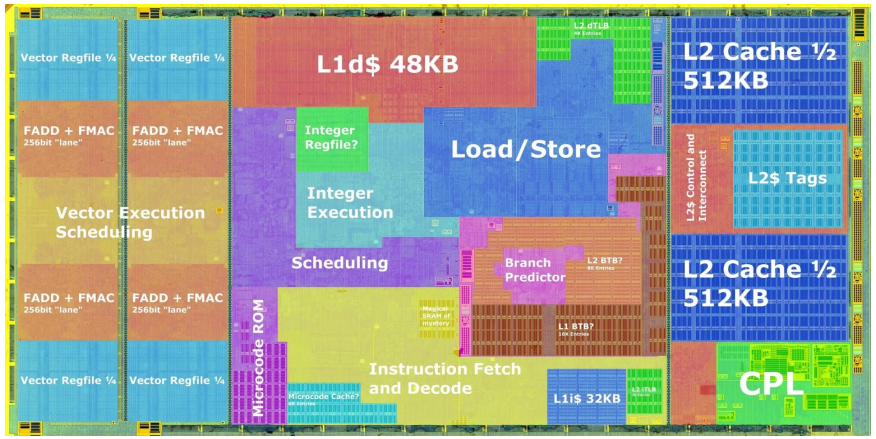
- Intel Core i9 (2022)
  - 16 cores
    - 8 performance-cores
      2-way SMT (HT)
      Up to 3.4 GHz
      10 MB L2 cache
    - 8 efficient-cores
      No HT
      Up to 4 GHz
    - Alderlake microarchitecture
  - 30MB shared L3
  - 10nm technology
  - Power consumption between 65W and 241W



More Examples

- AMD Ryzen 9000 (2024) Gen 5
  - 8 cores
  - 512-bit FP data path (AVX-512)
  - 32KB L1 instruction cache, 48KB L1 data cache
  - 1MB L2 cache per core
  - 32MB L3 cache
  - Includes TSVs to support stacked 3D V-cache
  - 4nm technology
  - 65W





CMP Issues

CMP vs. Multi-chip Multiprocessors

- While conceptually similar to traditional multiprocessors, CMP's have specific issues
  - Off-chip memory bandwidth: memory bandwidth typically grows more slowly than the number of cores
  - Shared memory hierarchy: cores compete for last-cache cache capacity and bandwidth, while hardware must maintain coherence
  - On-chip interconnection network: wires and metal layers are a very scarce resource
  - Wire delays: communication latency depends on physical distance and network contention
  - Power consumption and heat dissipation: All cores share the power-delivery and cooling limits of a single chip package

CMP vs. Multi-chip Multiprocessors

◎ While conceptually similar to traditional multiprocessors, CMP's have specific issues

- **Off-chip memory bandwidth: memory bandwidth typically grows more slowly than the number of cores**
- **Shared memory hierarchy: cores compete for last-cache cache capacity and bandwidth, while hardware must maintain coherence**
- On-chip interconnection network: wires and metal layers are a very scarce resource
- **Wire delays: communication latency depends on physical distance and network contention**
- Power consumption and heat dissipation: All cores share the power-delivery and cooling limits of a single chip package

The Memory Hierarchy

- Multi-core chips
  - L1 caches private
  - L2 caches private in some architectures and shared in others
  - Additional L3 or L4 caches

- Memory is always shared

- Deep cache hierarchy with large capacity to reduce memory traffic

Private vs. Shared Caches

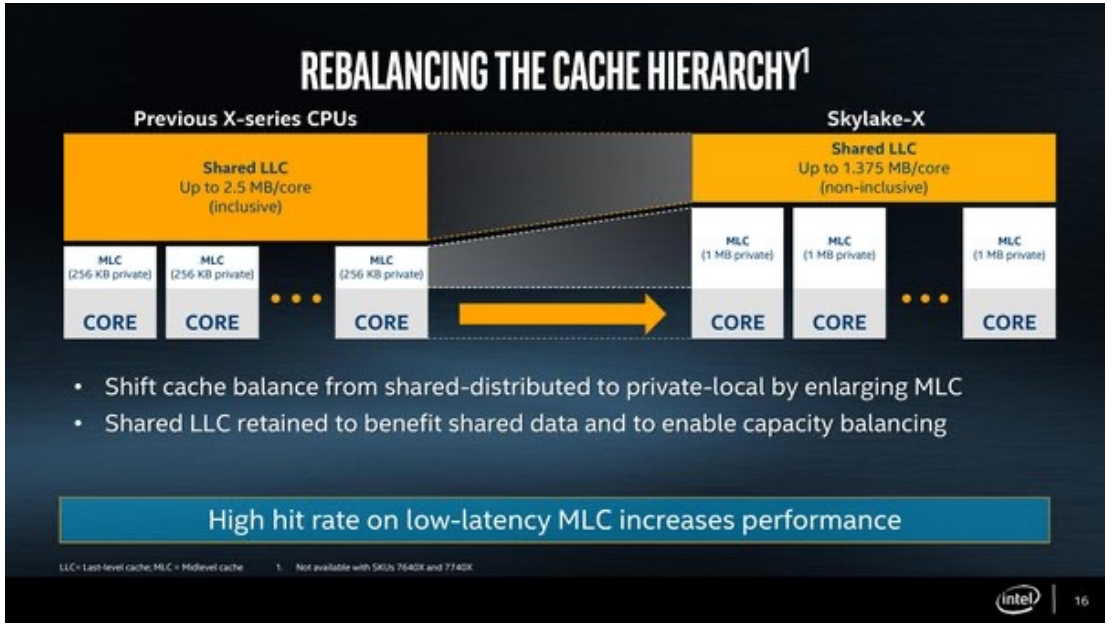
- Private caches
  - Are closer to core, so faster access
  - Reduces contention
- Shared caches
  - Threads on different cores can share the same cache data
  - More cache space available if a single (or a few) high-performance thread runs on the system
- Shared last-level cache (LLC) on a chip is where coherence is maintained across cores
  - Need to guarantee that all processors see a consistent value (i.e., consistent updates) for the same memory location

Private vs. Shared L2 Caches

- L1 caches are tightly integrated into the pipeline and are inseparable part of the core
- Private L2 caches
  - Less chance of negative interference between cores
  - Simpler interconnects
  - Possibly wasted storage is less loaded parts of the chip
  - Must enforce coherence across L2's
- Shared L2 caches
  - Possible positive interference between cores
  - Better utilization of storage
  - No need to enforce coherence
  - More chance for negative interference between cores
  - All-to-one interconnect takes up large area and may become a bottleneck

Caches in Modern CMPs

- Private L1 and L2 caches, shared L3 caches
- Bigger L2 caches, smaller L3 caches
  - With a higher number of cores, cache contention goes up. Making caches private helps dealing with it
- Replaces a ring bus with a mesh-style bus
  - A ring bus is not very scalable with a large number of cores



https://www.techpowerup.com/review/intel-core-i9-7900x/3.html

NUCA LLC Caches

- Large on-chip last-level caches are partitioned into independently accessible banks and subbanks
- Physical distance creates nonuniform access latency
  - A core can access nearby banks faster than distant banks
  - The same bank may be close to one core but far from another
- Non-uniform cache architecture (NUCA) exploits these latency differences
  - Frequently accessed cache lines can be mapped or migrated closer to the requesting core
  - Dynamic placement can adjust the balance between shared-cache capacity and private-cache locality
- Tradeoff
  - Dynamic mapping and migration improve locality but add lookup, data-movement, metadata, and replacement overhead

Summary

- CMP exploits both ILP and TLP by making each core simpler and providing a high level of coarse-grained parallelism
- CMP needs efficient cache hierarchy and on-chip interconnect to maximize parallelism while minimizing data transfer and communication overheads
- Programming and OS support is essential for writing parallel algorithms and managing thread scheduling

Vector Computers

Problems with Superscalar Approach

◎ Limitations of conventional ILP

- Clock frequency: Deeper pipelines can support higher frequencies but increase pipeline overhead and branch-misprediction penalties
- Instruction issue width: limited ILP reduces actual instruction issue/completion rate
- Cache hit rate: Many applications have very large working sets or continuous data streams, thus poor locality (poor memory latency hiding)
- Data parallelism: Scalar ILP mechanisms cannot efficiently exploit the regular array parallelism common in HPC and machine learning. This requires SIMD, vector, or matrix execution support

→ As a result, actual achieved performance is much less than peak potential performance and energy efficiency

Data Parallelism

- Concurrency arises from performing the same operation on different pieces of data
  - Single instruction multiple data (SIMD)
  - E.g., dot product of two vectors

- Contrast with data flow
  - Concurrency arises from executing different operations in parallel (in a data driven manner)

- Contrast with thread ("control") parallelism
  - Concurrency arises from executing different threads of control in parallel

- SIMD exploits operation-level parallelism on different data
  - Same operation concurrently applied to different pieces of data
  - A form of ILP where instruction happens to be the same across data

Vectorizable Applications

◎ Applications with a high degree of data parallelism (loop-level parallelism) suitable for vector processing

- Scientific computing
  - Computational chemistry, fluid dynamics, physics, astrophysics, ….
- Computer vision
- Bioinfomatics
- Cryptography
- Compression
- Databases
- …

Loop-Level Parallelism (LLP)

- Data parallelism: similar independent/parallel computations on different elements of arrays
→ independent (or parallel) loop iterations when implemented as a sequential program
- Exploit data parallelism among independent iterations of a loop

```
for (i=1;i<1000;i++) {
    x[i] = x[i] + y[i];
}
```

```
LV
LV
ADDV
SV
```

4 vector instructions:
Load Vector X
Load Vector Y
Add Vector X, X, Y
Store Vector X

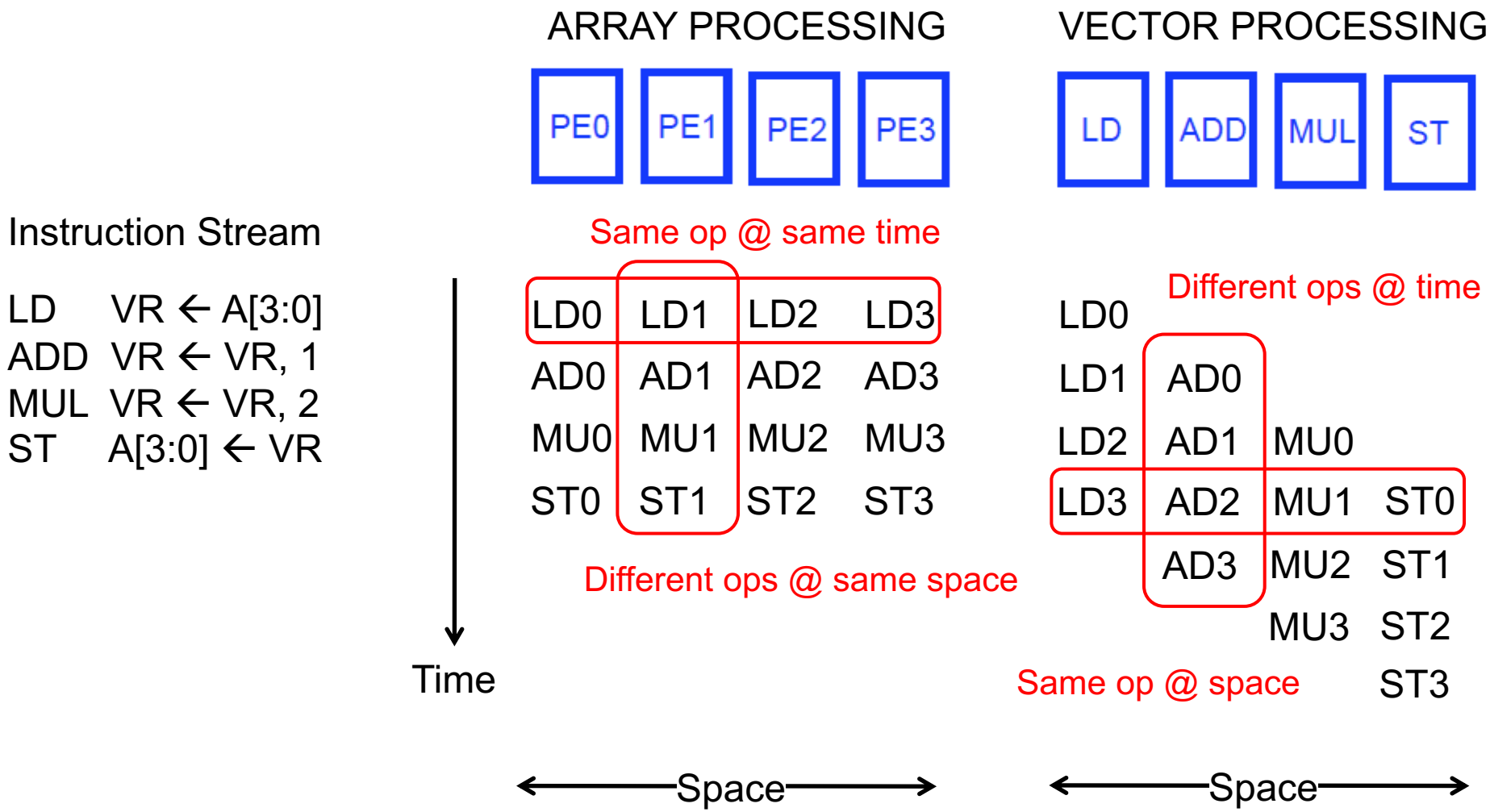
- In scientific computing, LLP has been traditionally exploited by vector ISAs

SIMD Processing

- Single instruction operates on multiple data elements
  - In time or in space
- Multiple processing elements

- Time-space duality
  - Array processing: Instruction operates on multiple data elements at the same time using different space
  - Vector processing: instruction operates on multiple data elements in consecutive time steps using the same space
  - Most modern SIMD processors are a combination of both
    - They exploit data parallelism in both time and space
    - GPUs are a prime example

Array vs. Vector Processors



Vector Processors (I)

- A vector is a one-dimensional array of numbers
- Many scientific/commercial programs use vectors

```
for (i = 0; i<=49; i++)
    C[i] = (A[i] + B[i]) / 2
```

- A vector processor is one whose instructions operate on vectors rather than scalar (single data) values
- A vector instruction performs an operation on each element in consecutive cycles
  - Vector functional units are deeply pipelined
  - Each pipeline stage operates on a different data element
  - E.g., memory latency only once per vector load or store vs. once per element → amortize the latency over multiple elements

Vector Processors (II)

- Each element in a vector instruction is independent of previous results
  - Multiple pipelined functional units (lanes) usually used

- Vector instructions allow deeper pipelines
  - No intra-vector dependencies → no hardware interlocking needed within a vector
  - No control flow within a vector
  - Known stride allows easy address calculation for all vector elements
    - Enables prefetching of vectors into registers/cache/memory

Vector Processor Advantages

- No dependencies within a vector
  - Pipelining & parallelization work really well
  - Can have very deep pipelines, no dependencies!
- Each instruction generates a lot of work
  - Reduces instruction fetch bandwidth requirements
- Highly regular memory access pattern
- No need to explicitly code loops
  - Fewer branches in the instruction sequence

Vector Processor Advantages

- Works (only) if parallelism is regular (data/SIMD parallelism)
- Very inefficient if parallelism is irregular
  - How about searching for a key in a linked list?
- Memory (bandwidth) can easily become a bottleneck, especially if
  1. compute/memory operation balance is not maintained
  2. data is not mapped appropriately to memory banks

Vector Processing in Depth

Vector vs. Single-Issue Scalar Processors

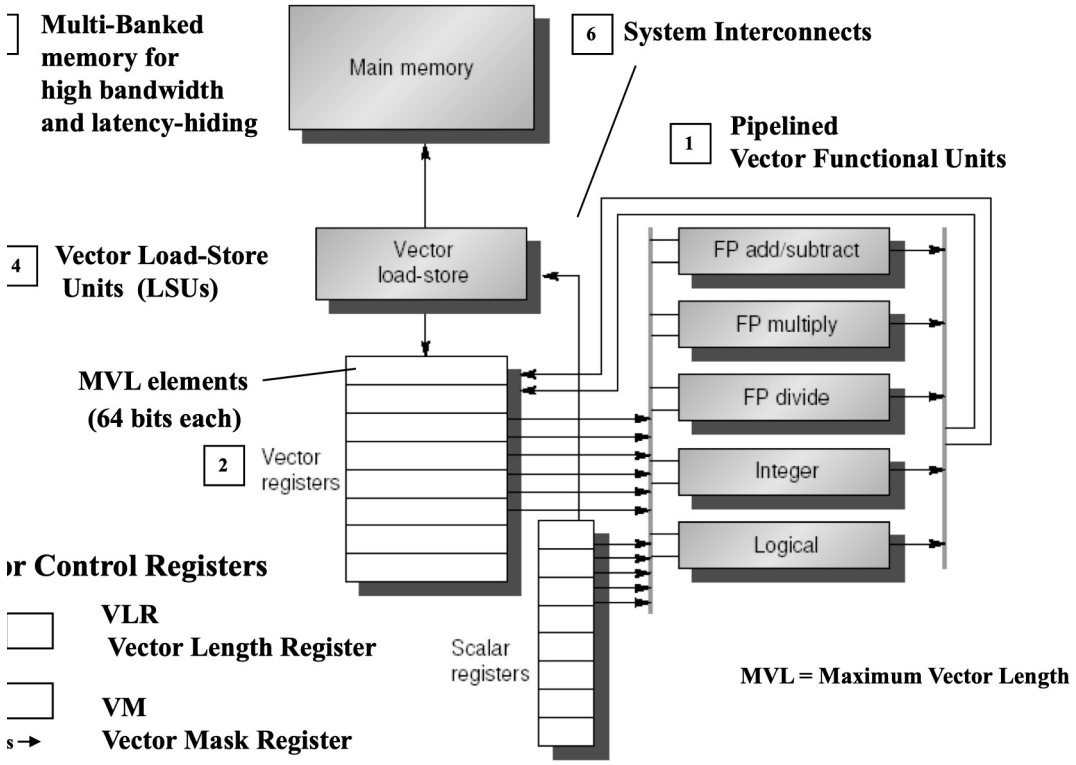
Single-Issue Scalar	Vector
One instruction fetch, decode, dispatch per operation	One instruction fetch, decode, dispatch per vector (up to maximum vector length)
Arbitrary register access, adds area and power	Structured register access
Loop unrolling and software pipelining for high performance increases i-cache footprint	Smaller code for high performance; less power in i-cache misses
All data passes through cache; waste power if no temporal locality	May bypass d-cache
One TLB lookup per load or store	One TLB lookup per group of loads or stores

Vector vs. Superscalar Processors

Superscalar	Vector
Control logic grows quadratically with issue width	Control logic grows linearly with issue width
Control logic consumes energy regardless of available parallelism	Vector unit switches off when not in use
Dynamic nature makes real-time performance less predictable	More predictable real-time performance
Speculation to increase visible parallelism wastes energy and adds complexity	Vector instructions expose data parallelism without speculation Software control of speculation when desired

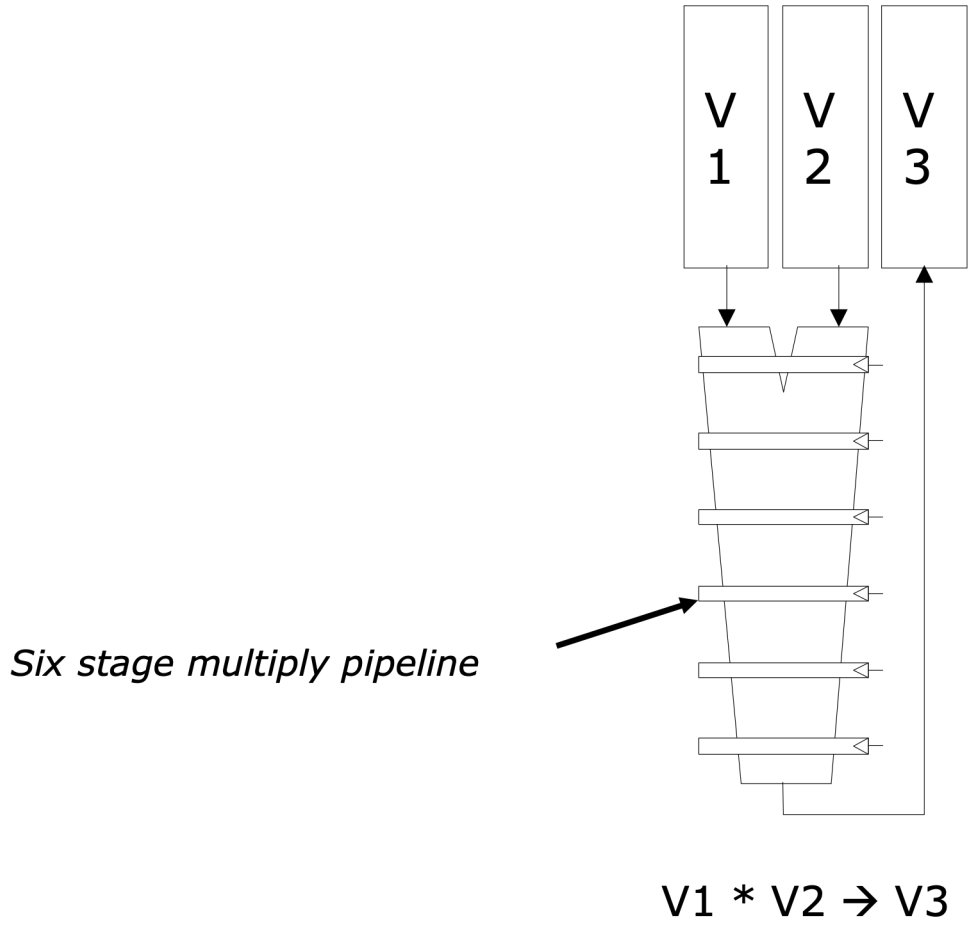
Basic Structure of Vector Register Architecture

- Includes a scalar unit and a vector unit
- A vector unit includes
  (1) Vector functional units (FUs or lanes)
  (2) Vector registers
  (3) Vector control registers
  (4) Vector memory Load-Store Units (LSUs)
  (5) Multi-banked main memory (for high bandwidth)
  (6) System component interconnects between scalar and vector processors



Vector Functional Units

- Fully pipelined, start new operation every clock
- Use a deep pipeline to execute element operations
  → fast clock cycle
- Control of deep pipeline is simple because elements in vector are independent
- Vector vs. scalar processors
  - Vector processor greatly reduces the dynamic instruction bandwidth
  - No overhead instructions (branches, index computations)
  - Vector instruction stalls only for the first element in each vector
    - C.f., every scalar instructions stall due to data hazards
- Vector maximizes advantages of pipelining and avoid its downsides



Vector Functional Units

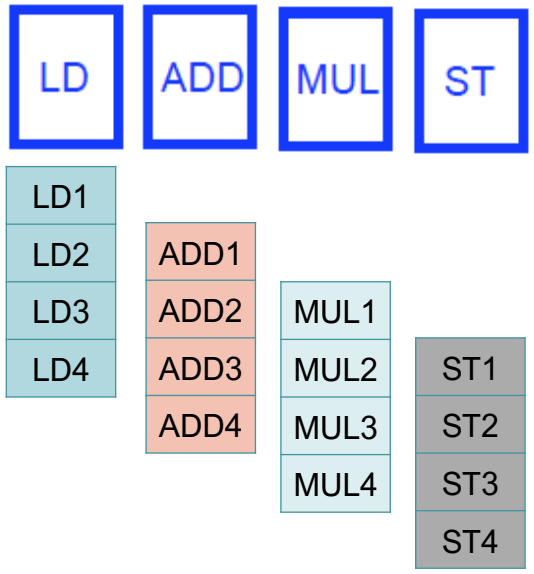
Instruction Stream (Vector)

```
LD    VR ← A[3:0]
ADD   VR ← VR, 1
MUL   VR ← VR, 2
ST    A[3:0] ← VR
```

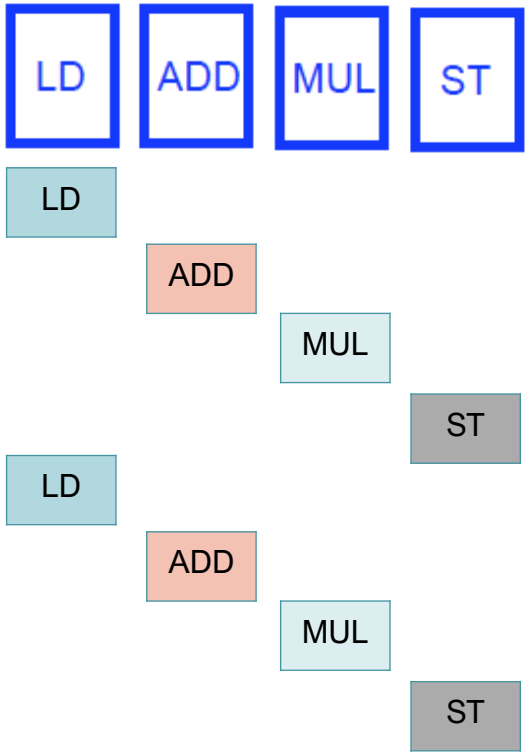
Instruction Stream (Scalar)

```
LD    R ← A[0]
ADD   R ← R, 1
MUL   R ← R, 2
ST    A[0] ← R
LD    R ← A[1]
ADD   R ← R, 1
MUL   R ← R, 2
ST    A[1] ← R
  …
```

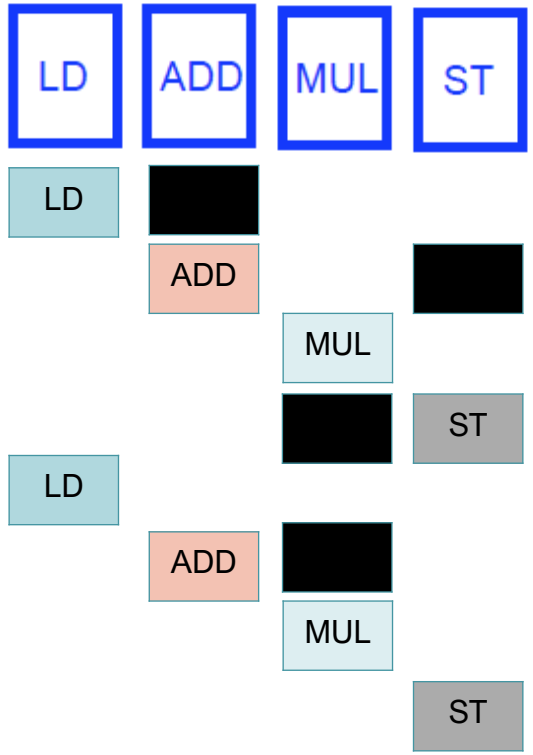
VECTOR PROCESSING



SCALAR PROCESSING

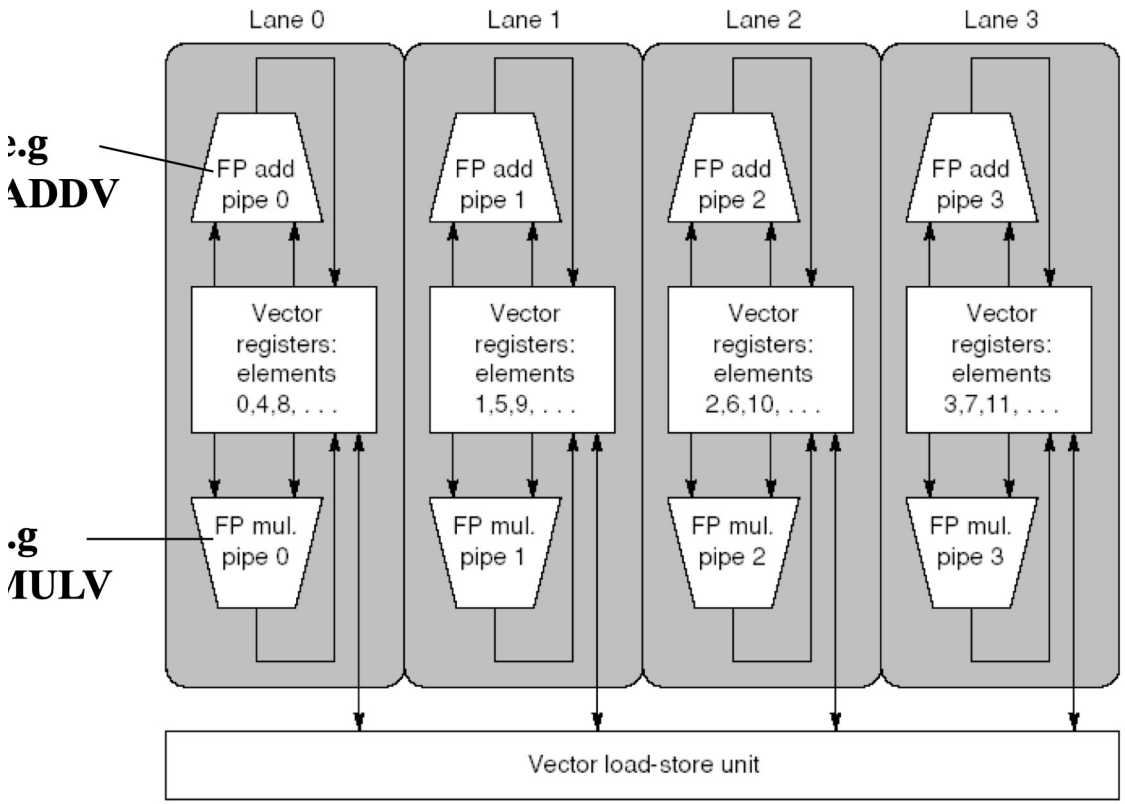


SUPERSCALAR OUT-OF-ORDER PROCESSING



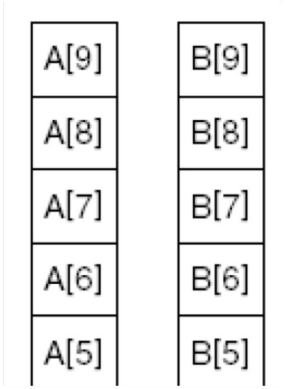
Multiple Lanes for Performance

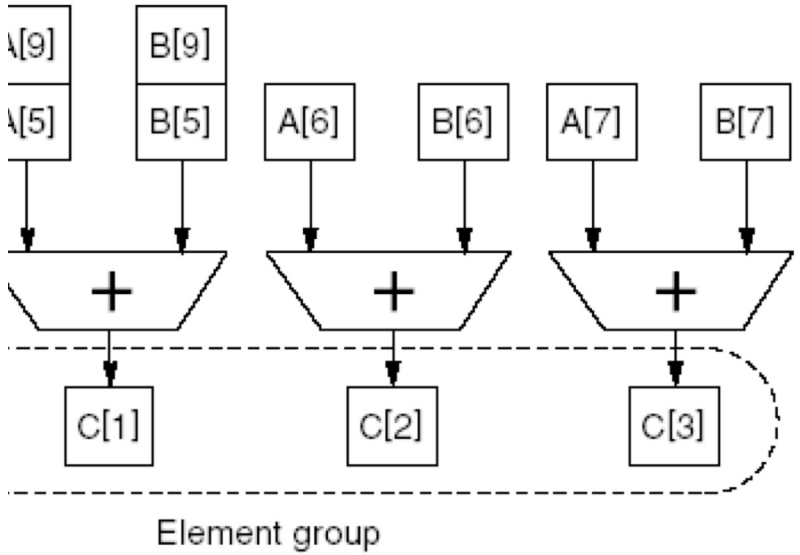
- Beyond one element per clock cycle
  - Use parallel pipelines to execute a vector instruction
- Each lane contains one portion of the vector register file and one execution pipeline from each vector functional unit
- Number of lanes in a vector unit = the # of vector FUs of the same type that are each assigned a number of computations of the same vector instruction
- Requires little increase in control complexity



Multiple Lanes for Performance

One lane: 10 cycles + startup

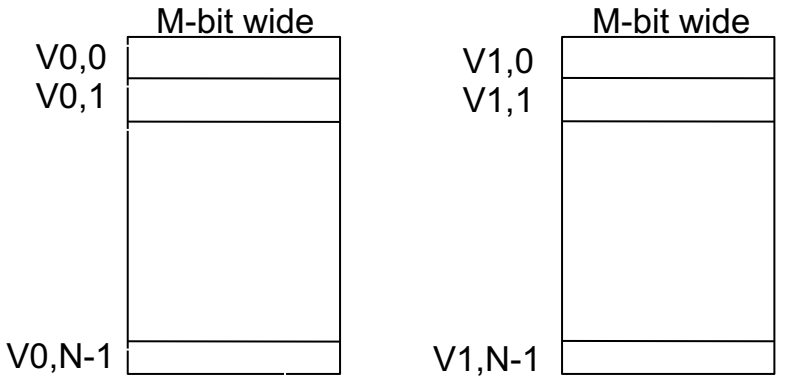




Four lanes: 3 cycles + startup

Vector Registers

- Each vector data register holds N elements of M-bit values
  - Contains a fixed number of vector registers(typically 8-32)
- Vector control registers: VLEN, VSTR (stride), VMASK
- Maximum VLEN can be N
  - Maximum number of elements stored in a vector register
- Fixed-width SIMD CPUs encode VLEN in instructions
  - Maximum number of elements determined by the register width and element size
- Vector Mask Register (VMASK)
  - Indicates which elements of vector to operate on
  - Set by vector test instructions
    - e.g., VMASK[i] = (V_k[i] == 0)



Gather/Scatter Operations

◎Want to vectorize loops with indirect accesses:

```
for (i=0; i<N; i++)
 A[i] = B[i] + C[D[i]]
```

◎Indexed load instruction (*Gather*)

```
LV vD, rD         # Load indices in D vector
LVI vC, rC, vD    # Load indirect from rC base
LV vB, rB         # Load B vector
ADDV.D vA,vB,vC   # Do add
SV vA, rA         # Store result
```

Gather/Scatter Operations

- Gather/scatter operations often implemented in hardware to handle sparse vectors (matrices)
- Vector loads and stores use an index vector which is added to the base register to generate the addresses
- *Scatter* example

Index Vector	Data Vector (to Store)
0	3.14
2	6.5
6	71.2
7	2.71

Stored Vector (in Memory)	
Base+0	3.14
Base+1	X
Base+2	6.5
Base+3	X
Base+4	X
Base+5	X
Base+6	71.2
Base+7	2.71

Conditional Operations in a Loop

◎ What if some operations should not be executed on a vector (based on a dynamically-determined condition)?

```
loop:       for (i=0; i<N; i++)
                if (a[i] != 0) then b[i]=a[i]*b[i]
```

◎ Idea: Masked operations

- VMASK register is a bit mask determining which data element should not be acted upon

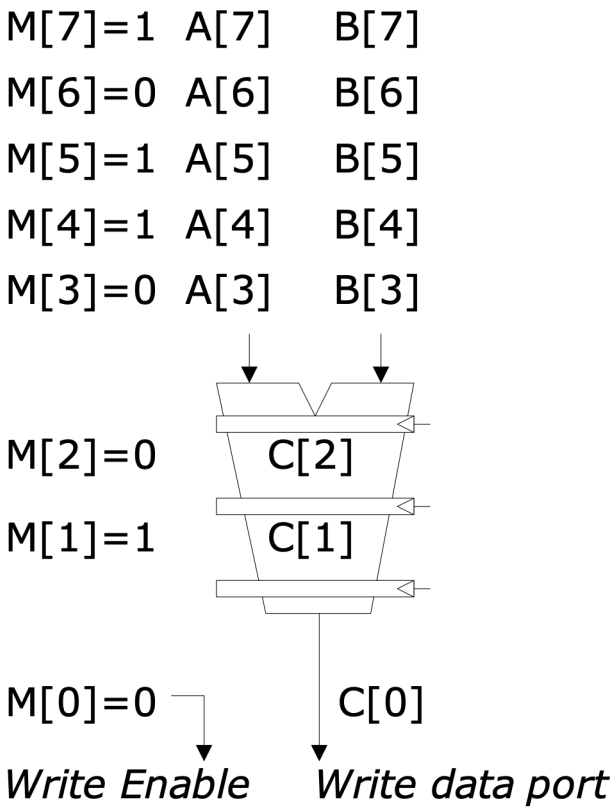
```
VLD V0 = A
VLD V1 = B
VMASK = (V0 != 0)
VMUL V1 = V0 * V1
VST B = V1
```

- This is predicated execution. Execution is *predicated* on mask bit

Masked Vector Instructions

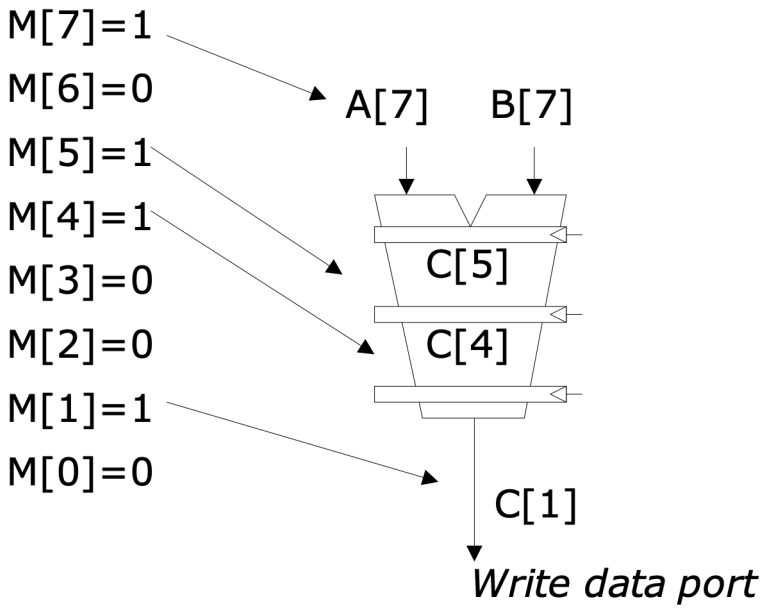
Simple Implementation

- execute all N operations, turn off result writeback according to mask



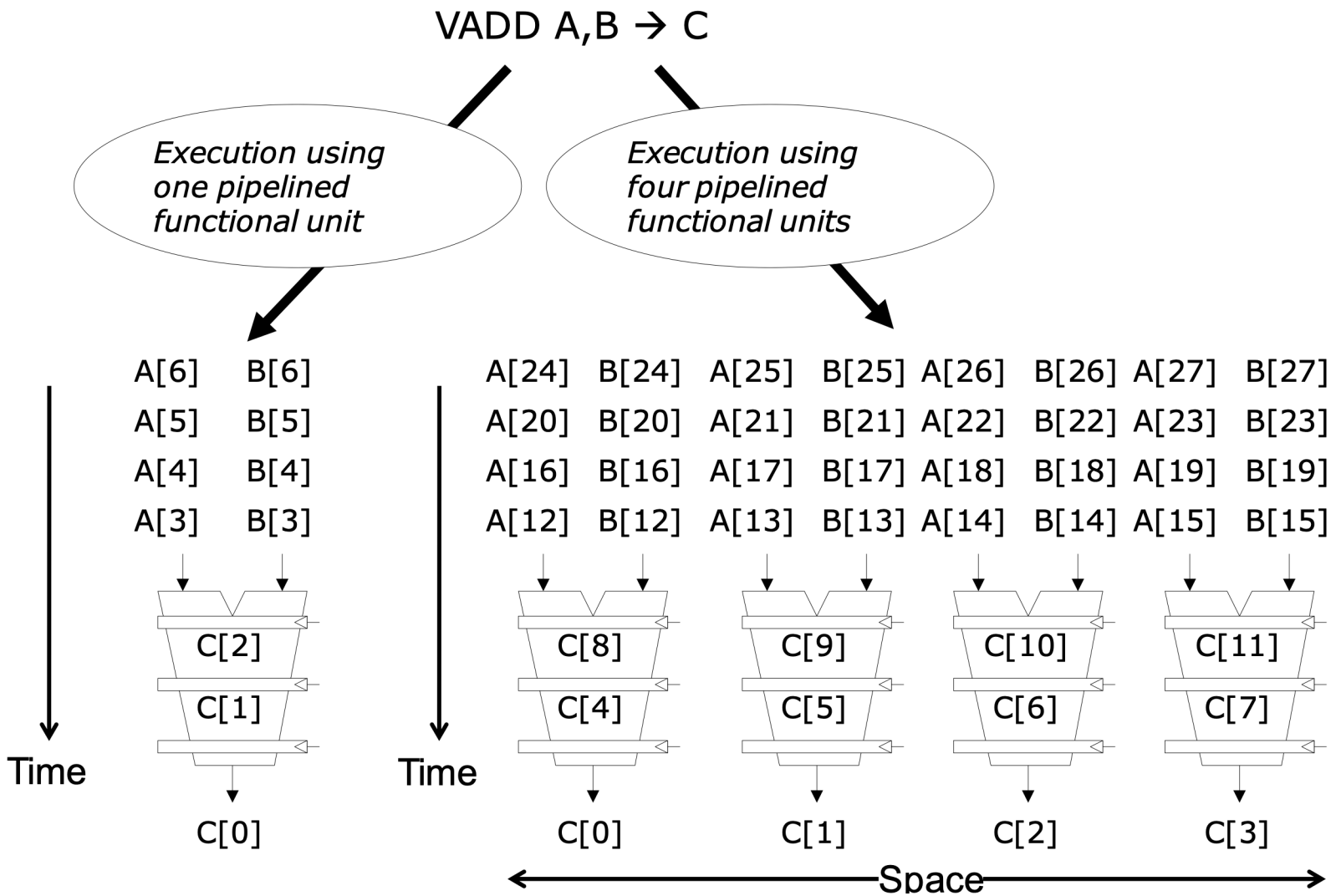
Density-Time Implementation

- scan mask vector and only execute elements with non-zero masks

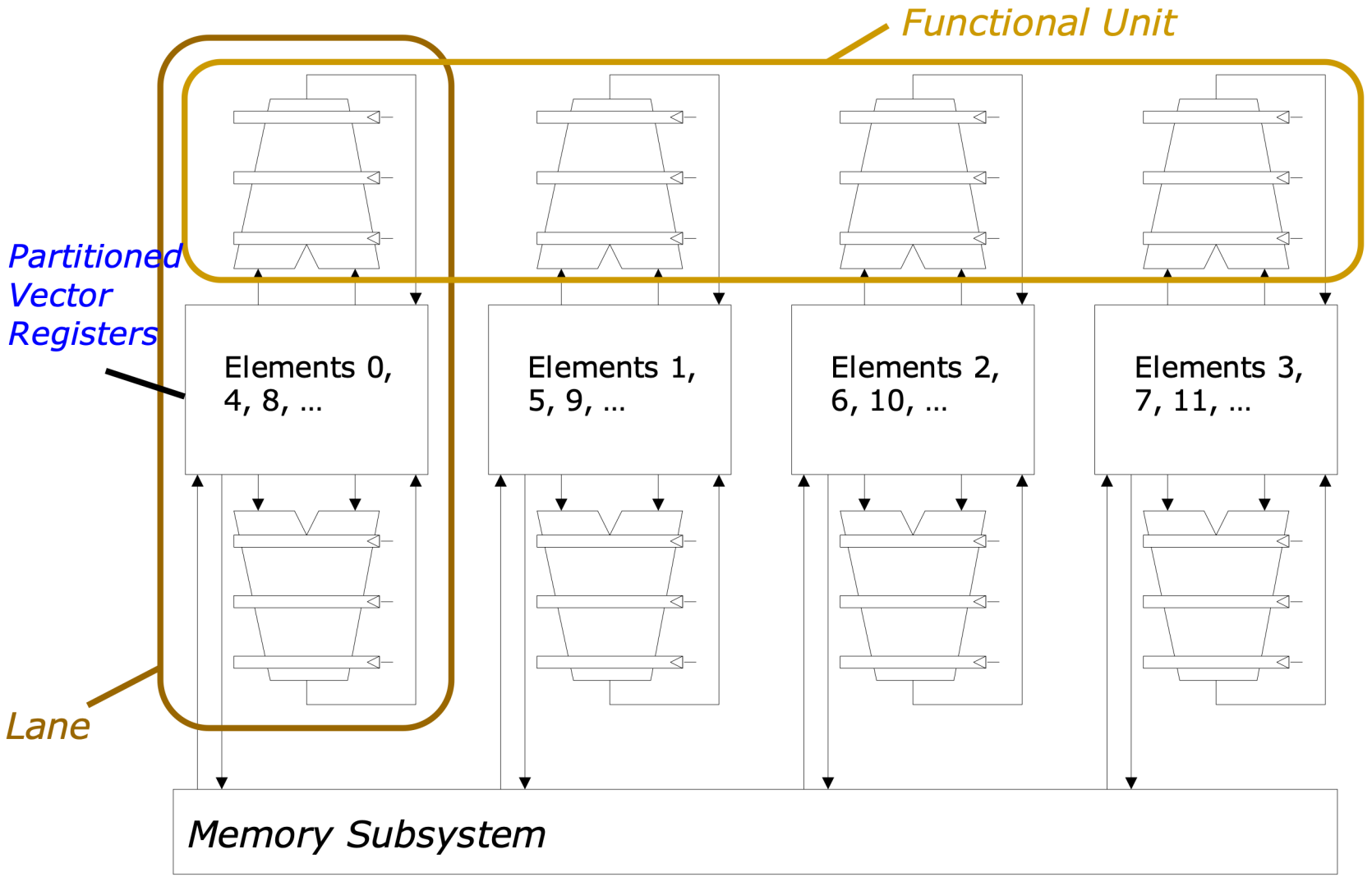


Slide credit: Krste Asanovic

Vector Instruction Execution

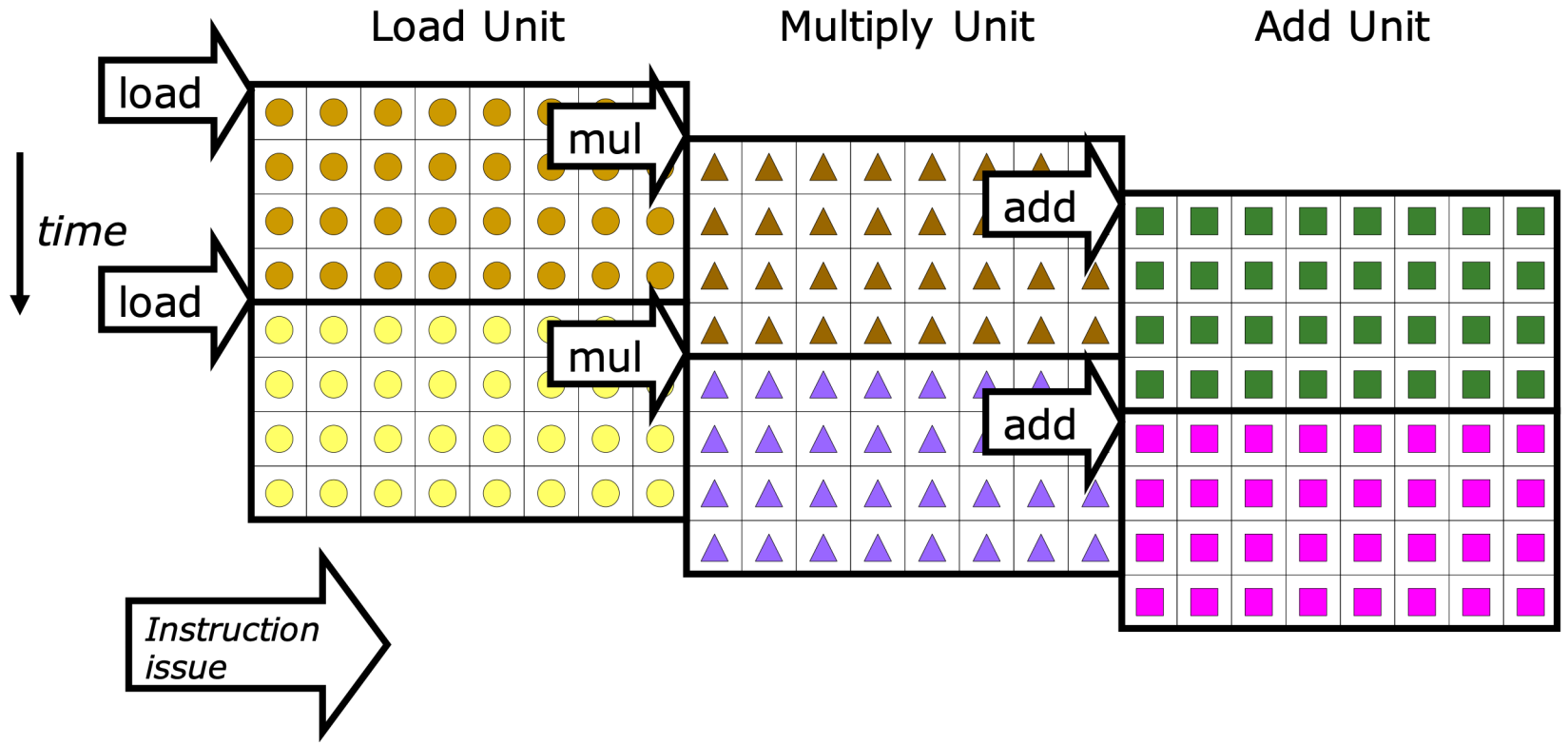


Vector Unit Structure



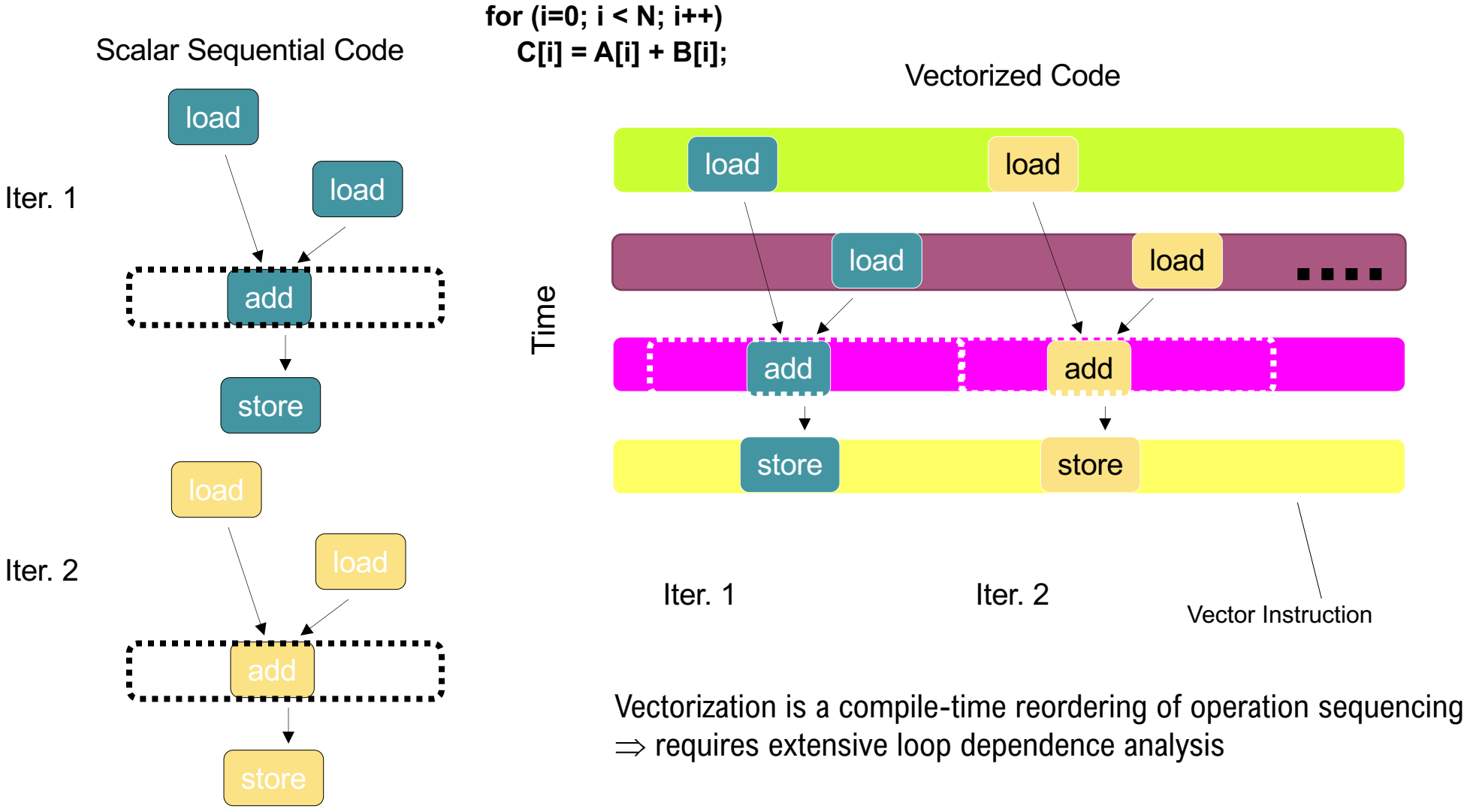
Vector Instruction Level Parallelism

- Can overlap execution of multiple vector instructions
  - Example machine has 32 elements per vector register and 8 lanes
  - Completes 24 operations/cycle while issuing 1 vector instruction/cycle



Automatic Code Vectorization

```
for (i=0; i < N; i++)
    C[i] = A[i] + B[i];
```



Vectorization is a compile-time reordering of operation sequencing
⇒ requires extensive loop dependence analysis

Slide credit: Krste Asanovic

Vector/SIMD Processing Summary

- Vector/SIMD machines are good at exploiting regular data-level parallelism
  - Same operation performed on many data elements
  - Improve performance, simplify design (no intra-vector dependencies)

- Performance improvement limited by vectorizability of code
  - Scalar operations limit vector machine performance
  - Remember Amdahl's Law

- Many existing ISAs include (vector-like) SIMD operations
  - Intel MMX/SSEn/AVX, PowerPC AltiVec, ARM Advanced SIMD

SIMD Operations in Modern ISA

Vector Computers vs. SIMD Engines

Vector Computers	SIMD Engines
Dedicated vector processor	Specialized functional unit in CPU
Vector length is dynamic and determined in runtime	Vector length is fixed by hardware
Support high-bandwidth memory systems with support for strided and gather-scatter vector loads	Rely on aligned memory accesses and optimize for contiguous memory accesses C.f., modern vector-style variable-length engines (RISC-V Vector, ARM SVE)
May bypass d-cache; load large contiguous data directly from memory with special engine	Exploits d-cache for locality

Classic Vectors and Fixed-Width SIMD

- **Both apply one operation to multiple data elements**
  - Multiple lanes and pipelining exploit parallelism across space and time
  - An instruction may take several cycles, while independent instructions overlap
- **The key difference is how the element count is specified and provided**
  - Classic vector machines use an active vector length (up to a hardware limit)
  - Fixed-width SIMD instructions specify the operand width and element type, which together determine the element count

Vector computers

```
// MVL is maximum vector legnth
i = 0
while i < N:
    VL = min(MVL, N - i)
    SET_VECTOR_LENGTH(VL)
    va = VLOAD(A + i)
    vb = VLOAD(B + i)
    vc = VADD(va, vb)
    VSTORE(C + i, vc)
    i += VL
```

Fixed-width SIMD

```
W = 256 / 32        // 8 elements per instruction
i = 0
while i + W <= N:
    va = VLOAD_256_F32(A + i)
    vb = VLOAD_256_F32(B + i)
    vc = VADD_256_F32(va, vb)
    VSTORE_256_F32(C + i, vc)
    i += W
//Handle remaining elements
while i < N:
    C[i] = A[i] + B[i]
    i += 1
```

Classic Vectors and Fixed-Width SIMD

- **Both process long arrays in chunks**
  - Classic vectors can adjust the active length for the final chunk
    - MVL for classic supercomputers: 64 to 256 elements
    - No need for masking for the final chunk
  - Fixed-width SIMD handles remaining elements with scalar code or masked vector instructions

Vector computers

```
// MVL is maximum vector legnth
i = 0
while i < N:
    VL = min(MVL, N - i)
    SET_VECTOR_LENGTH(VL)
    va = VLOAD(A + i)
    vb = VLOAD(B + i)
    vc = VADD(va, vb)
    VSTORE(C + i, vc)
    i += VL
```

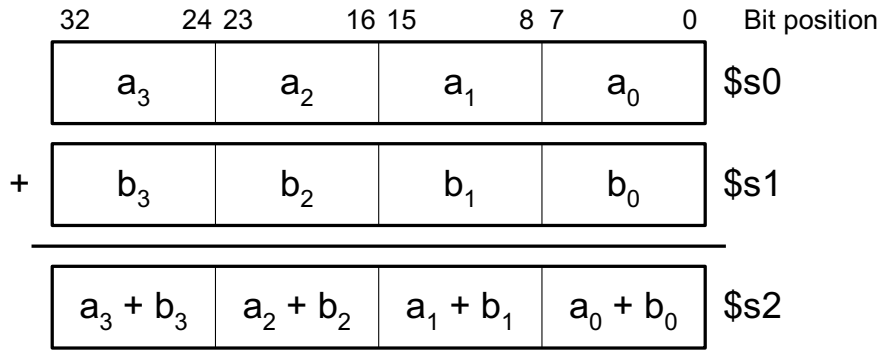
Fixed-width SIMD

```
W = 256 / 32        // 8 elements per instruction
i = 0
while i + W <= N:
    va = VLOAD_256_F32(A + i)
    vb = VLOAD_256_F32(B + i)
    vc = VADD_256_F32(va, vb)
    VSTORE_256_F32(C + i, vc)
    i += W
//Handle remaining elements
while i < N:
    C[i] = A[i] + B[i]
    i += 1
```

SIMD ISA Extensions

- Single Instruction Multiple Data (SIMD) extension instructions
  - Single instruction acts on multiple pieces of data at once
  - Common application: graphics, media processing, and numerical computation
  - Element count depends on register width and element size
- For example: add four 8-bit numbers
- Must modify ALU to eliminate carries between 8-bit values

```
padd8 $s2, $s0, $s1
```




```
// Semantics of one packed-add instruction
for each element k in {0, 1, 2, 3}:
    c[k] = (a[k] + b[k]) mod 256
```

SIMD Execution Inside a CPU Core

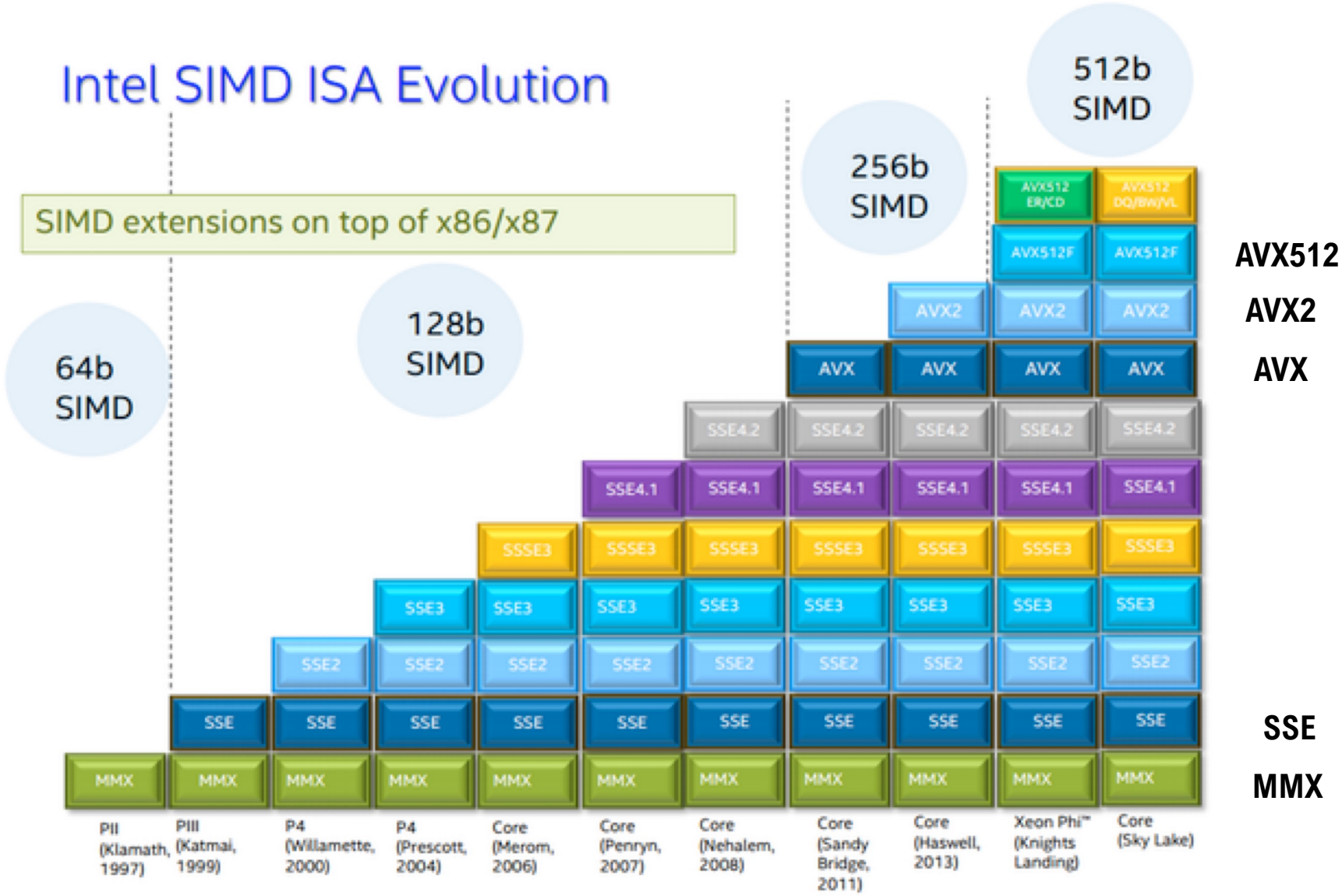
- **Scalar and SIMD instructions share one instruction stream**
  - A thread mixes address calculation, branches, scalar operations, and vector operations

- **The execution backend contains SIMD functional units**
  - On an out-of-order core, the scheduler issues ready instructions to available units
  - Independent scalar, SIMD, and memory operations may overlap

- **Vector loads and stores use the CPU memory hierarchy**
  - Load/store bandwidth, cache misses, and register pressure can limit SIMD throughput

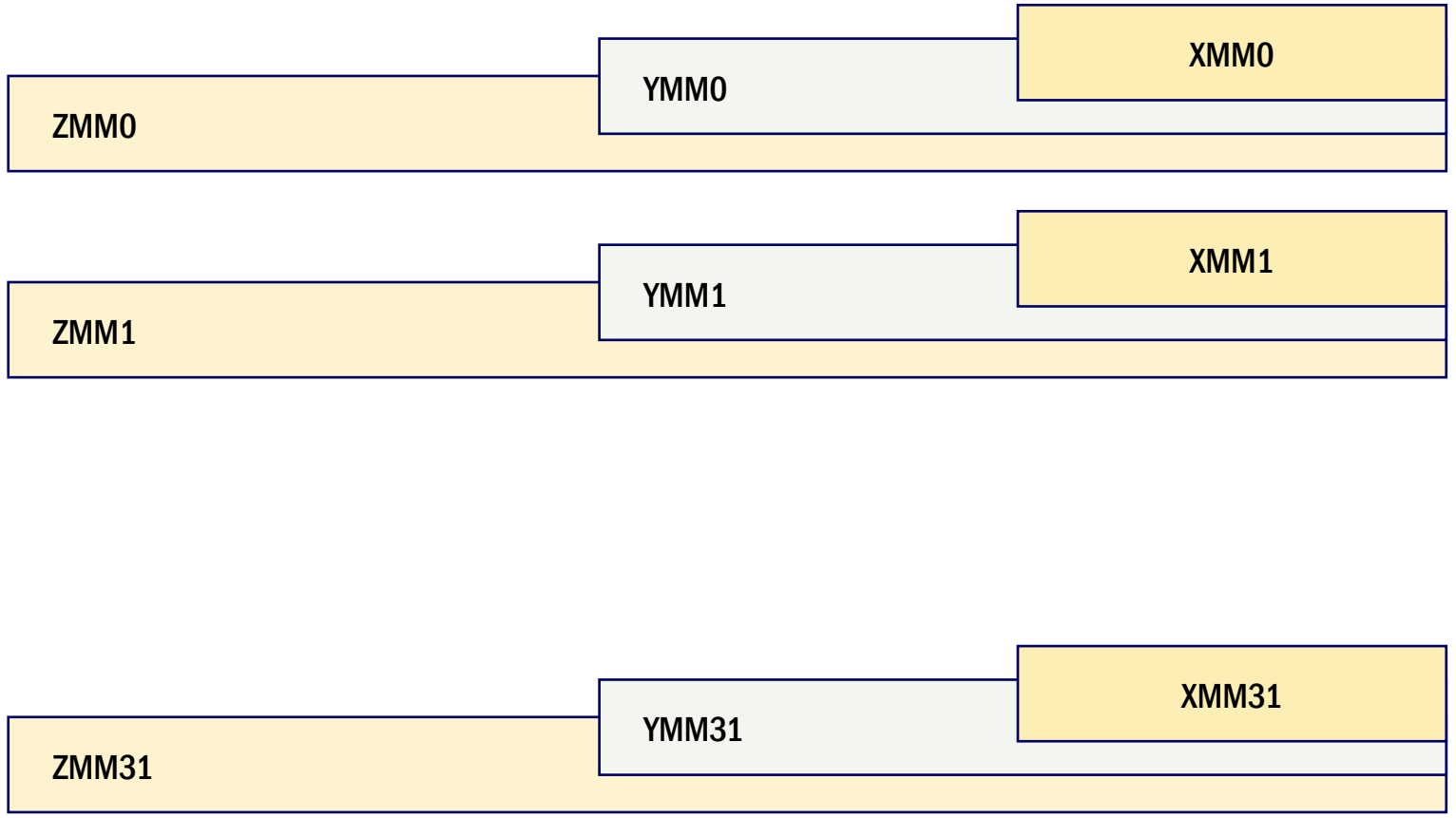
- **Multiple CPU threads can each use SIMD**
  - Thread parallelism and SIMD can be complementary

Intel SIMD Extensions

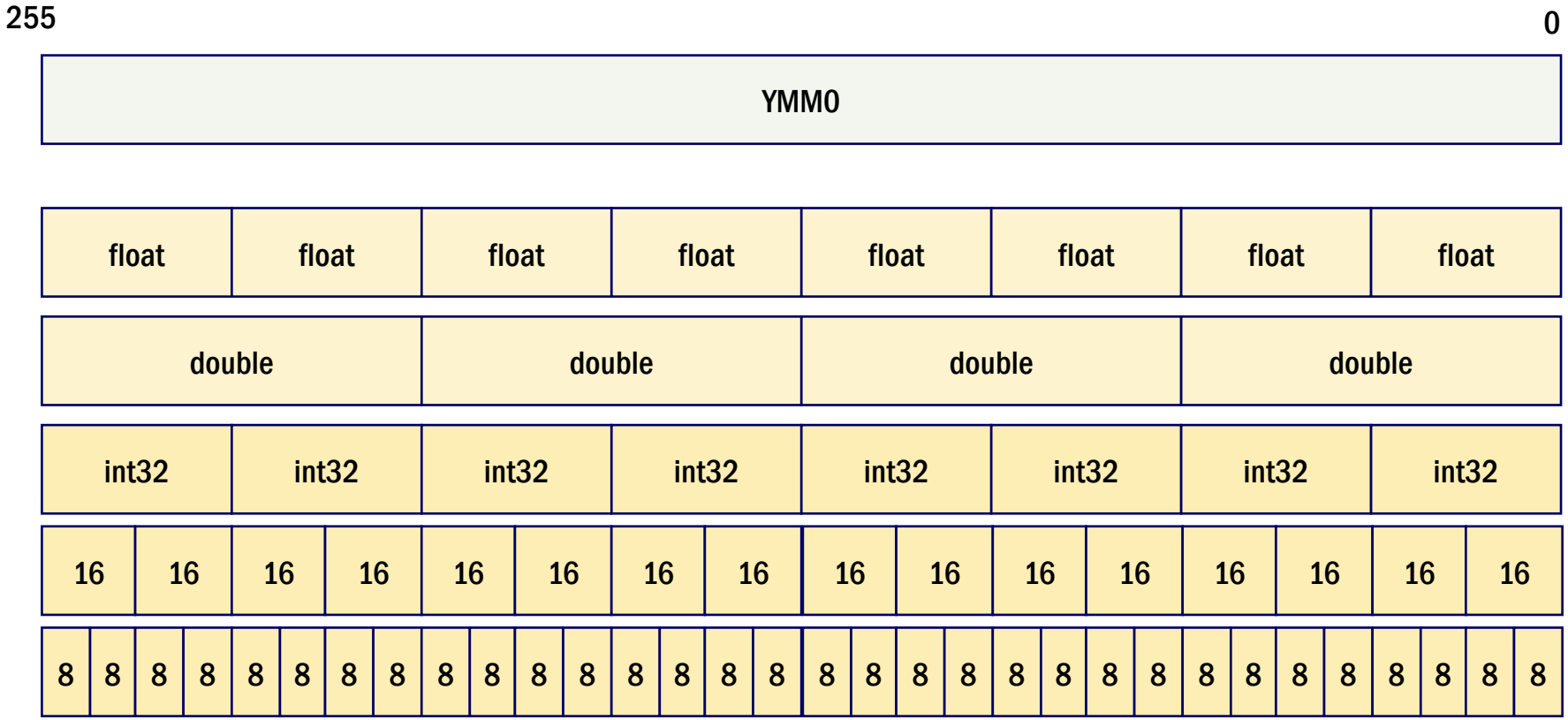


256-bit AVX/AVX2 and AVX-512

- XMM0 – XMM15
  - 128-bit registers
  - SSE
  - Lower half of YMM
- YMM0 – YMM15
  - 256-bit registers
  - AVX, AVX2
  - Lower half of ZMM
- ZMM0 – ZMM31
  - 512-bit registers
  - AVX-512



SSE/AVX Data Types



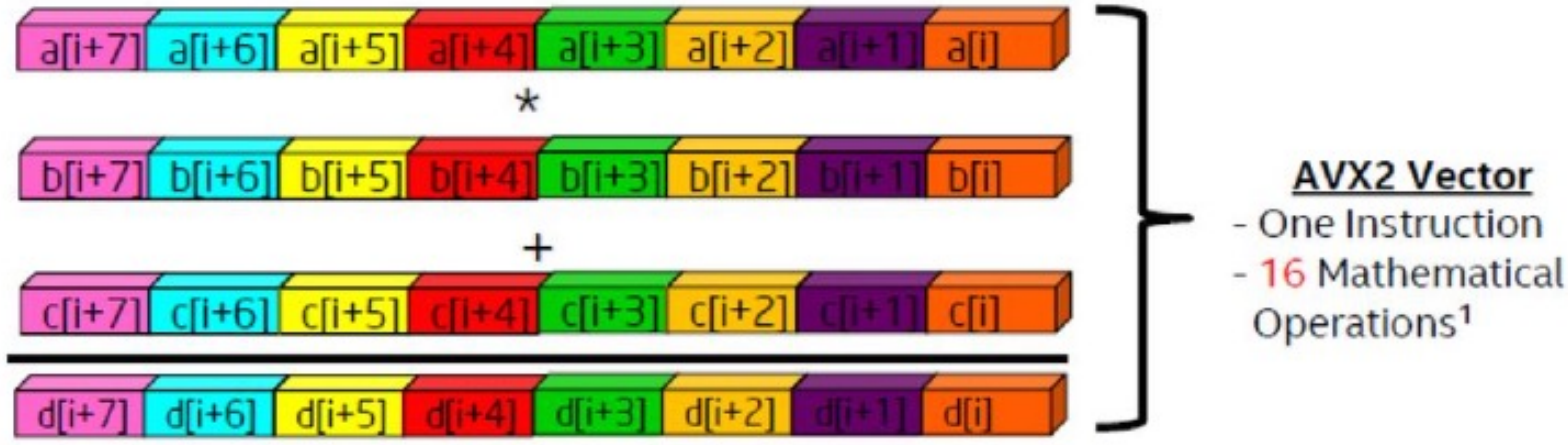
◎ Element count = vector width / element width

256-bit AVX/AVX2 Example

◎ Compute eight FP32 results with one vector FMA (Fused Multiply-Add) instruction

- Each element computes D[i] = A[i] * B[i] + C[i]

```
for (i=0;i<=MAX;i++)
   d[i]=((a[i]*b[i]) + c[i]);
```



1. Number of operations per instruction varies based on the which SIMD instruction is used and the width of the operands
8 of the operations are multiplications and 8 are additions (the addition of the multiplication result to a third operand)
1. 8 Multplication operations + 8 Addition multiplications

SIMD Intrinsics

- Intrinsics expose SIMD operations through C/C++ interfaces
  - Users can write explicit vector operations without inline assembly
  - Compiler handles register allocation
- Intrinsics Guide
  - https://software.intel.com/sites/landingpage/IntrinsicsGuide
  - https://support.arm.com/documentation/dui0472/m/ARMv6-SIMD-Instruction-Intrinsics

```
__m256 a, b, c;
__m256 d = _mm256_fmadd_ps(a, b, c); // d[i] = a[i]*b[i]+c[i] for i = 0 …7
```

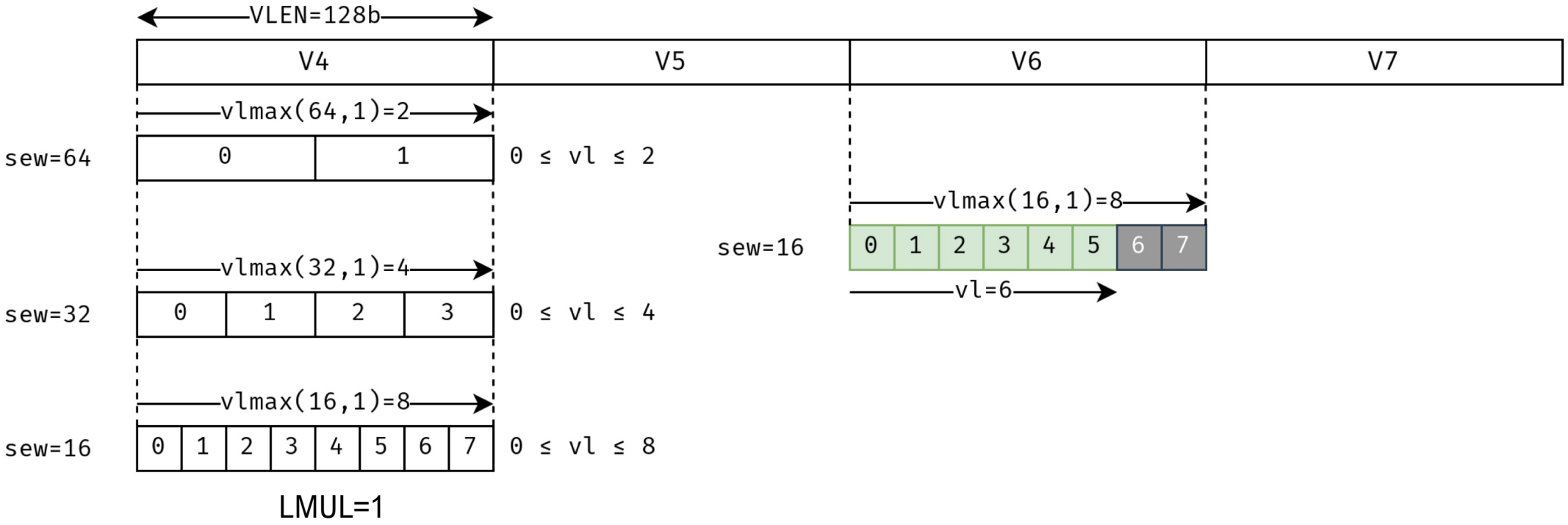
X86 AVX

```
float32x4_t a, b, c;
float32x4_t d = vfmaq_f32(c, a, b); // d[i] = c[i]+a[i]*b[i] for i = 0 … 3
```

ARM Neon

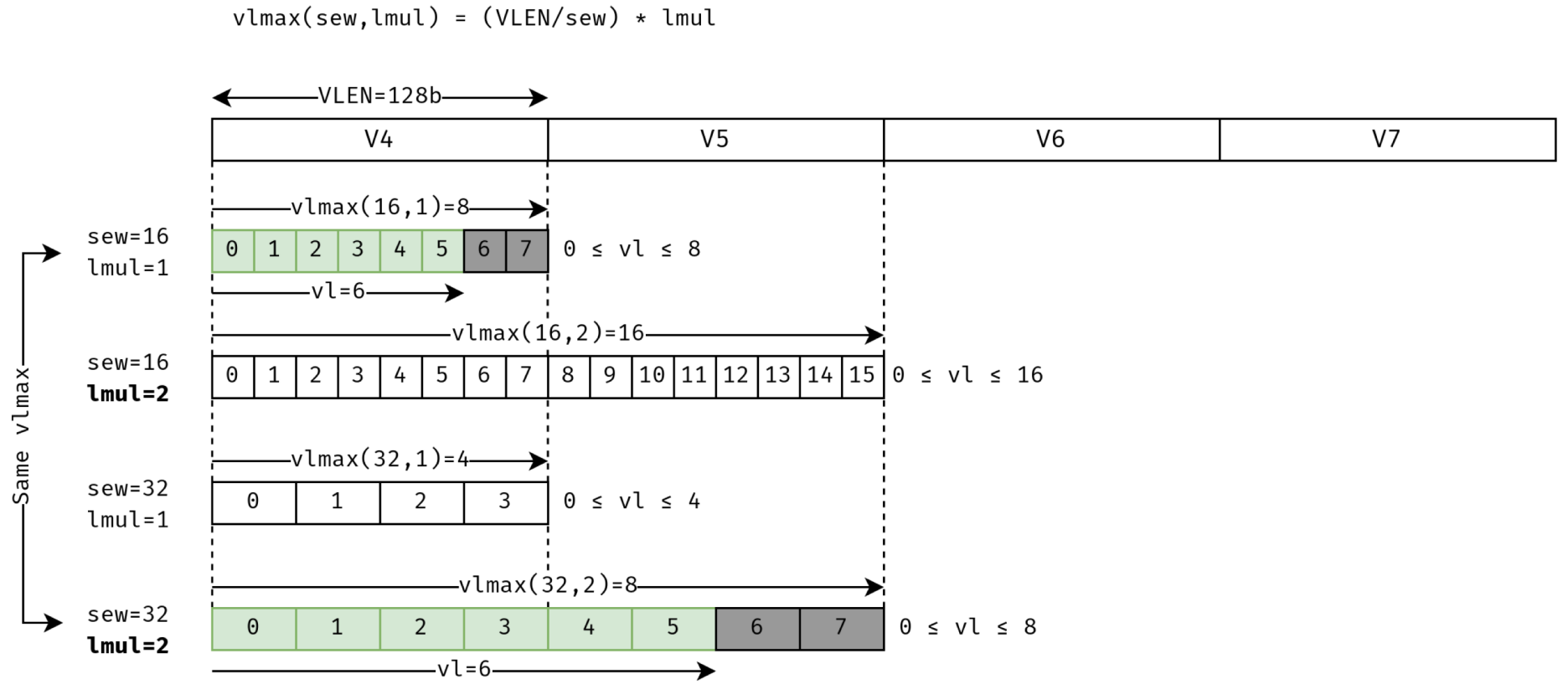
RISC-V Vector Extension

- RVV defines 32 vector registers (v0-v31) of size VLEN bits
- Vectors in RVV are divided in elements
  - SEW specifies the selected element width
- vl (Vector Length) register describes how many elements of the vector to operate



RISC-V Vector Extension

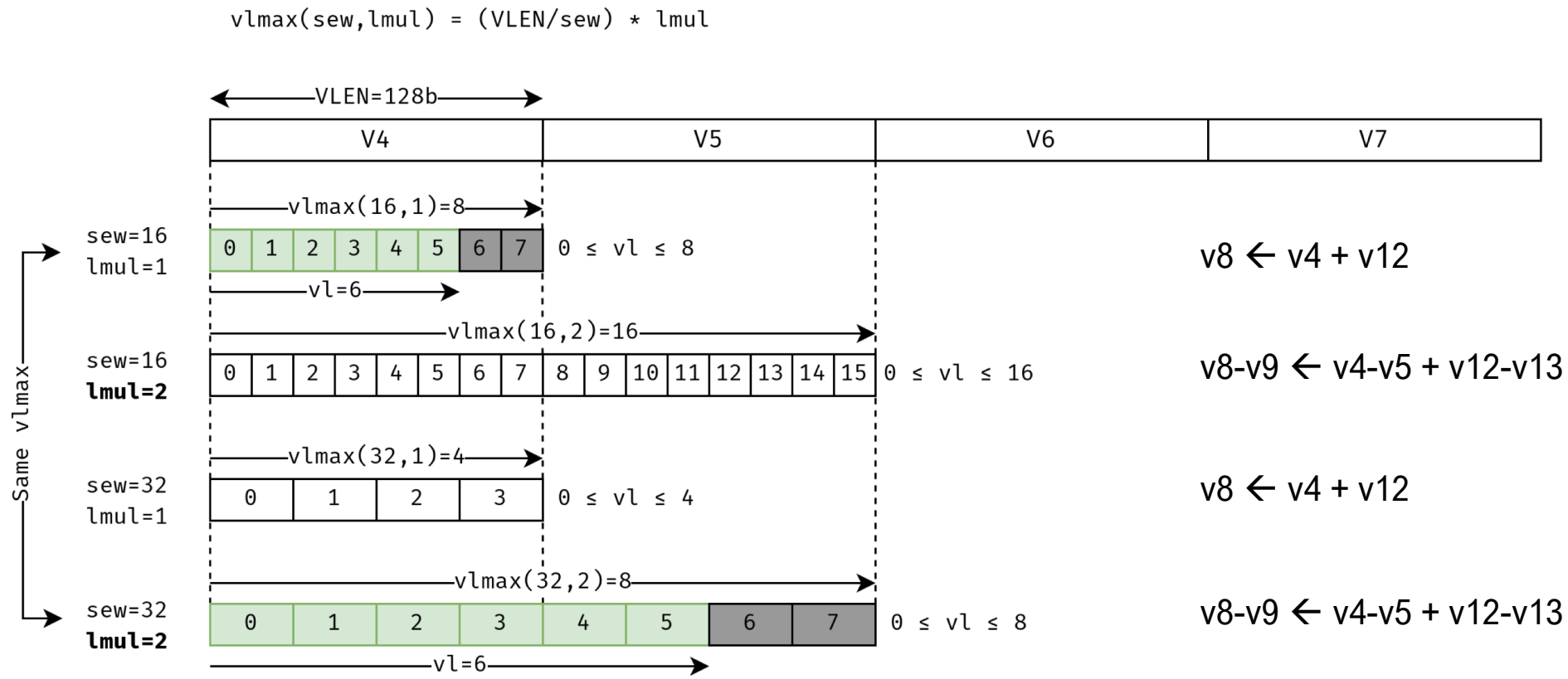
- Length multiplier (LMUL) allows operating on a fraction of a vector register or a group of vector registers
- Increasing LMUL increases capacity per operand



RISC-V Vector Extension

◎ The same vector instruction operates on different element widths and register groups

- vadd.vv v8, v4, v12



Acknowledgments

- Slides are adapted from and inspired by
  - Heterogeneous parallel computing at POSTECH, Multicore programming primer at MIT, Parallel computer architecture and programming at CMU, Parallel architecture at UIUC, Introduction to parallel computing at U of Chicago
- Intel and RISC-V sources
  - https://cdrdv2.intel.com/v1/dl/getContent/671368
  - https://eupilot.eu/wp-content/uploads/2022/11/RISC-V-VectorExtension-1-1.pdf

Next Class

GPU Architectures 1